CLAROTY

TEAM82

What We Learned From ICS Pwn2Own

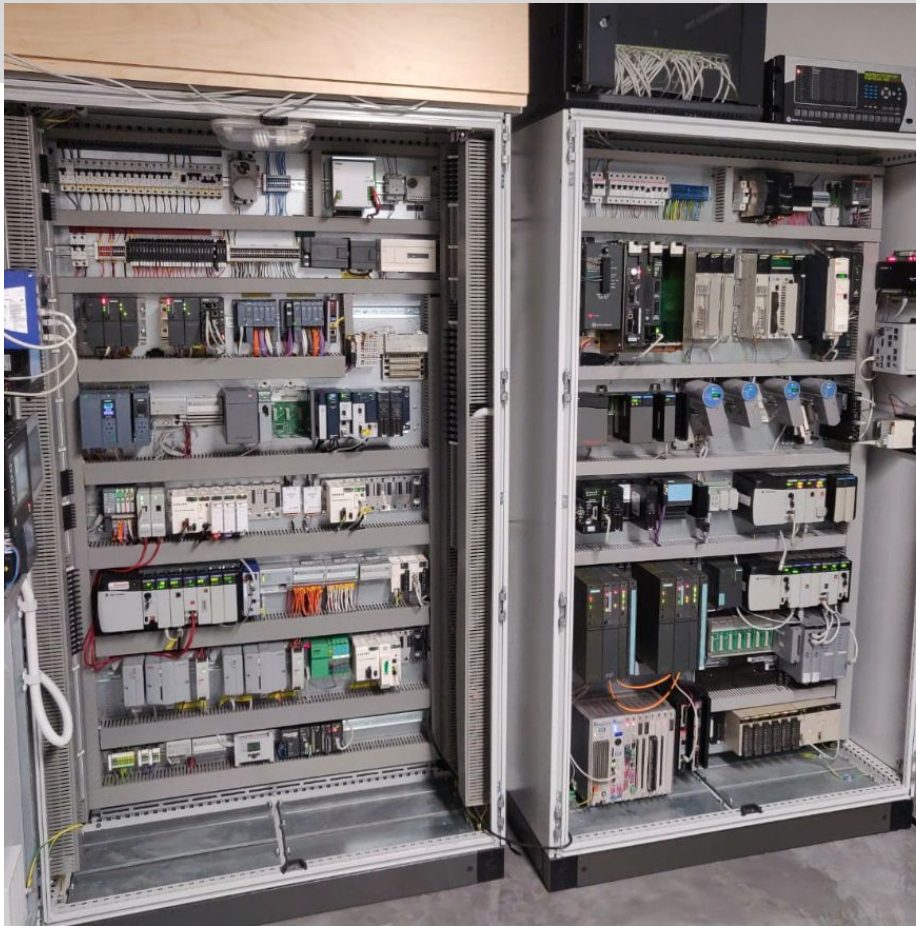
Sharon Brizinov @ Claroty Team82

$whoami

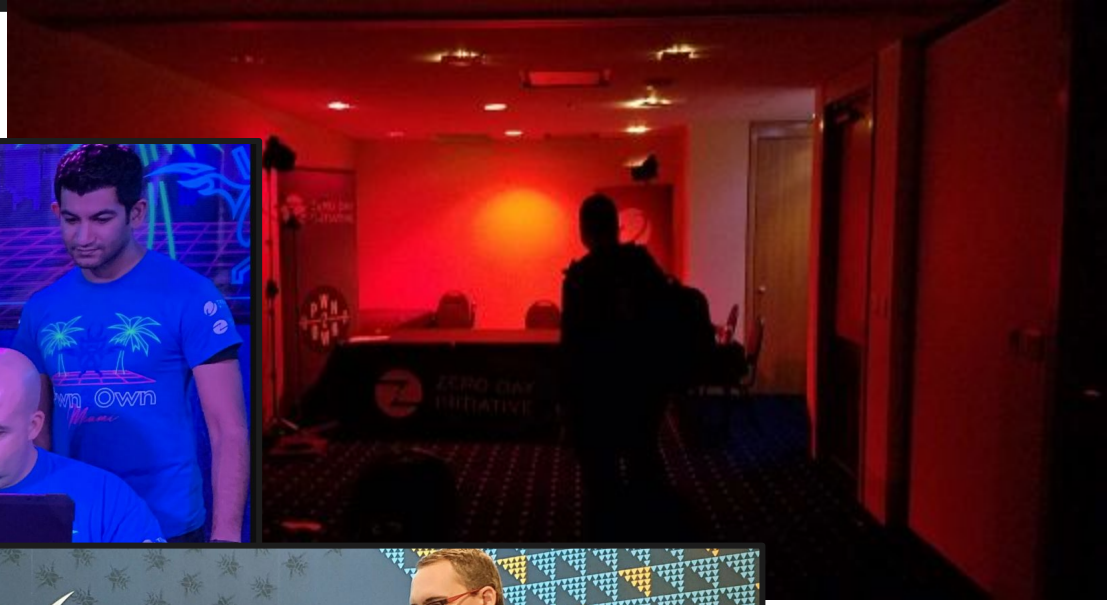


Sharon Brizinov

- OT Expert
- Vulnerability Researcher
- Specialize in Embedded Research and ICS/SCADA Security.









CLAROTY

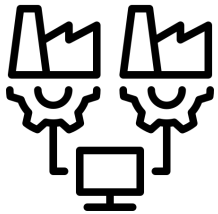

MIAMI
MASTER OF PWN 2023

Pwn2Own Events

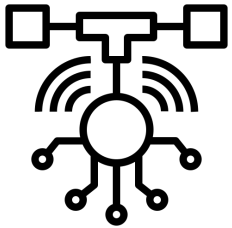


Pwn2Own ICS: Vendors and targets

ICONICS
Unified Automation
kepware
PROSYS OPC
Rockwell Automation
optimize!
Schneider Electric
AVEVA
inductive automation
OPC FOUNDATION
softing
TRIANGLE MICROWORKS, INC.
SOLUTIONS FOR COMMUNICATION PROTOCOL DEVELOPMENT



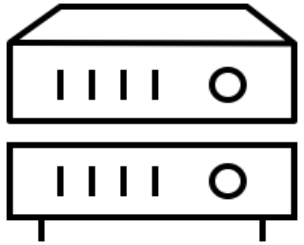
SCADA Server



ICS Protocols



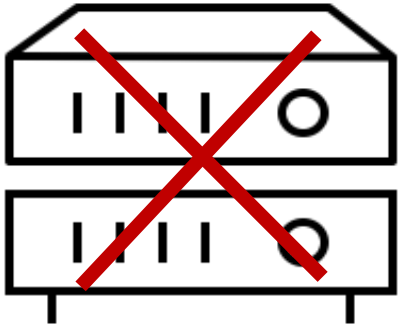
HMI

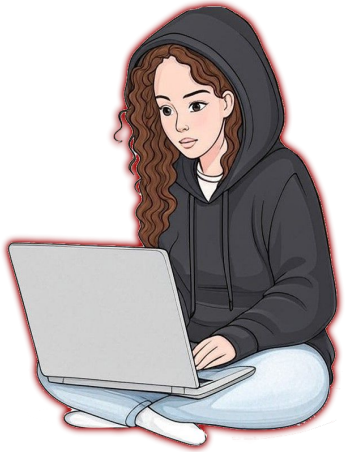


Engineering Workstation

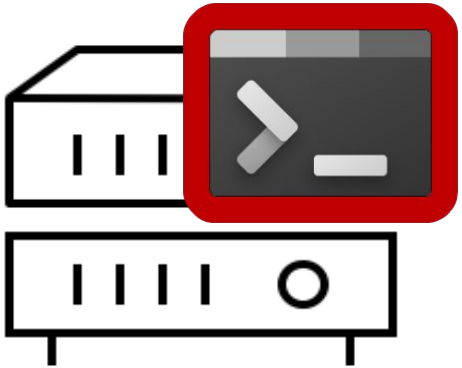
Pwn2Own ICS: Exploit Types

DOS





RCE



Exploit

~$5,000

Exploit

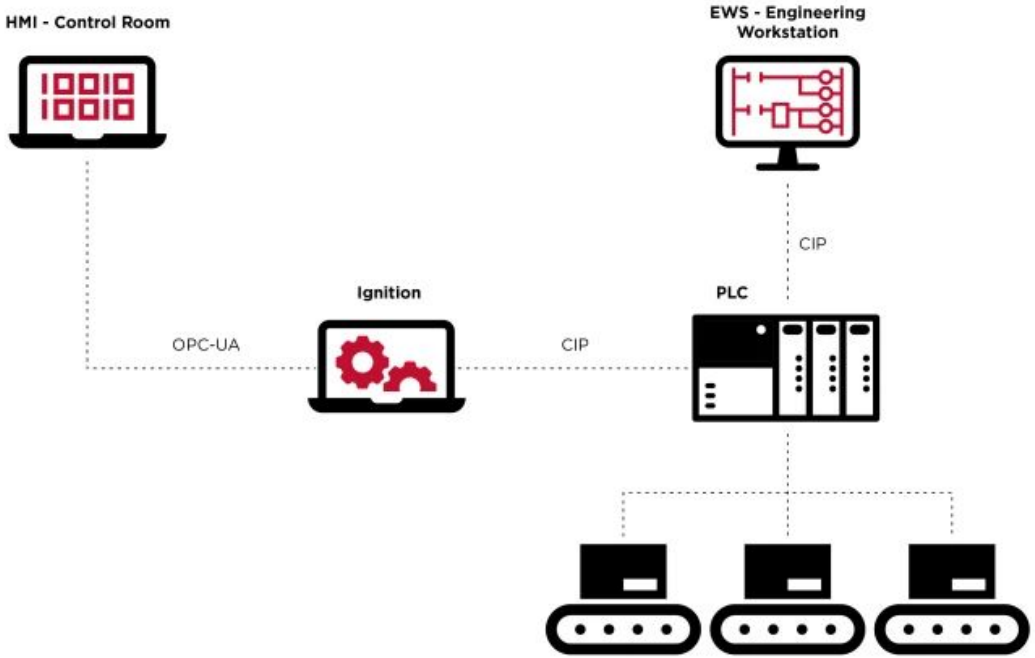
~$20,000

CLAROTY

Pwn2Own ICS Strategy

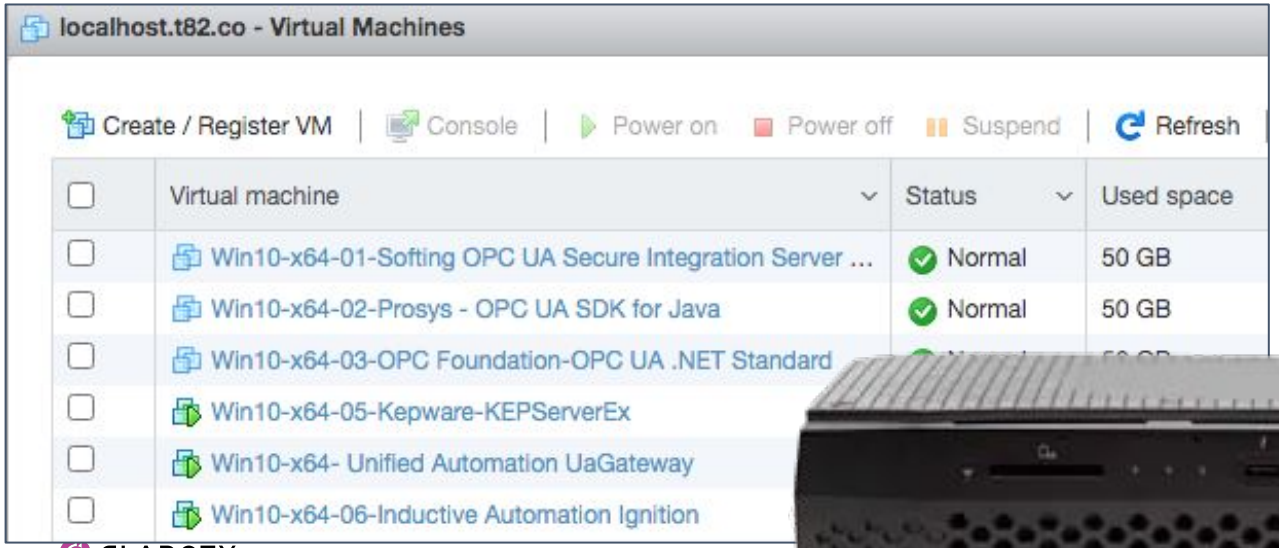
Pwn2Own Strategy

- Learn about ICS/OT/SCADA



Pwn2Own Strategy

- Learn about ICS/OT/SCADA
- Invest in setups: complex setups for **HMI, EWS, SCADA** software



*** Intel NUC X2**

*** Separate VM for each target**



Pwn2Own Strategy

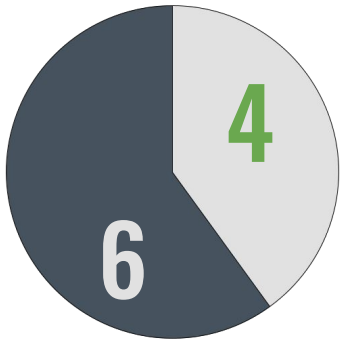
- Learn about ICS/OT/SCADA
- Invest in setups: complex setups for **HMI, EWS, SCADA** software
- ICS protocol stacks deep dive: **OPC-UA**, DNP3, ENIP, etc

OPC UA Targets

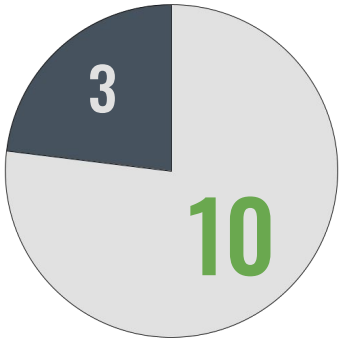
Non-OPC UA Targets



2020



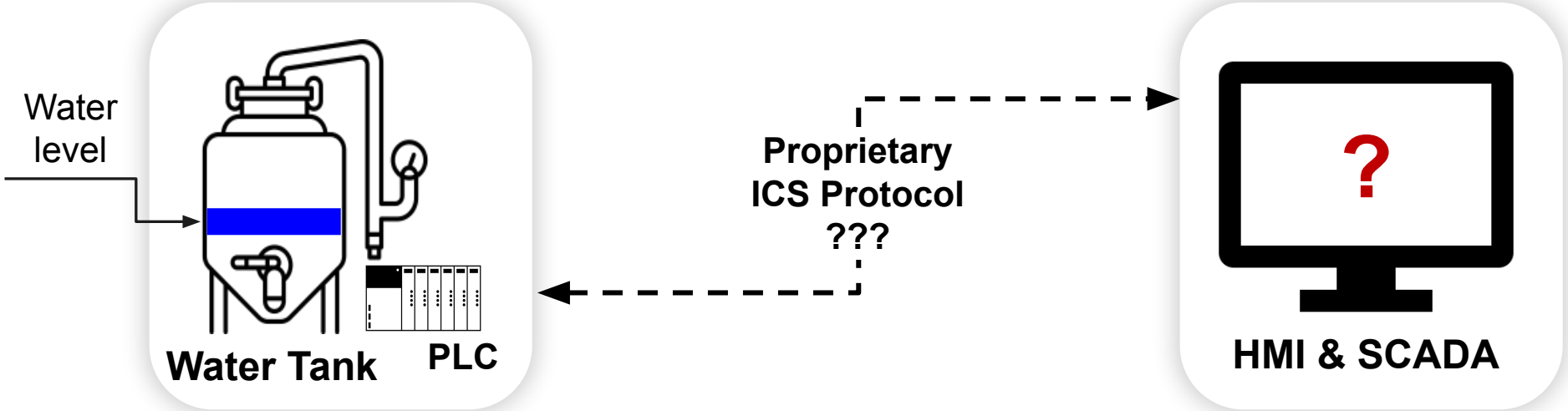
2022



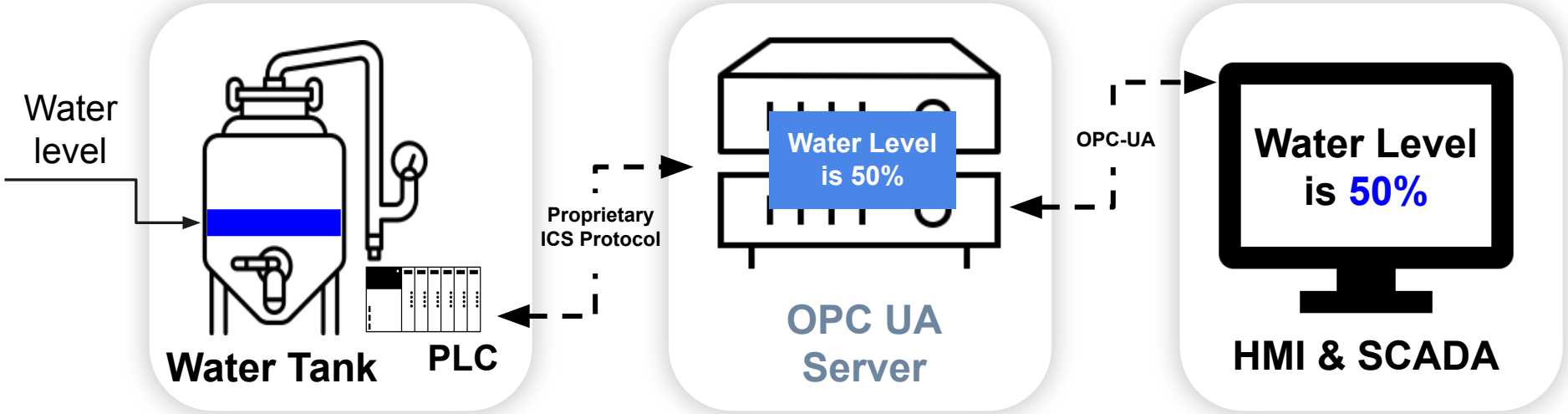
2023

Case Study : OPC UA Strategy

What is the OPC UA Protocol?



What is the OPC UA Protocol?



OPC UA Protocol: Key Features

- **Open Protocol:** OPC UA is a standard for sharing data in industrial systems.
- **Interoperable:** Connects devices and systems from different vendors.
- **Secure:** It protects data with encryption and access control.
- **Documentation:** vast and detailed documentation



OPC UA - Targets

UA Automation C++ Server	OPC Foundation OPC UA .NET	Prosys OPC UA SDK for JAVA	Softing Integration Server	KEPServerEx
Proprietary + OPC Foundation C++	OPC Foundation .NET	Proprietary + OPC Foundation Java	Proprietary + OPC Foundation C	Proprietary

OPC UA Protocol Research: Documentation Deep Dive

6.7.2.2 Message Header

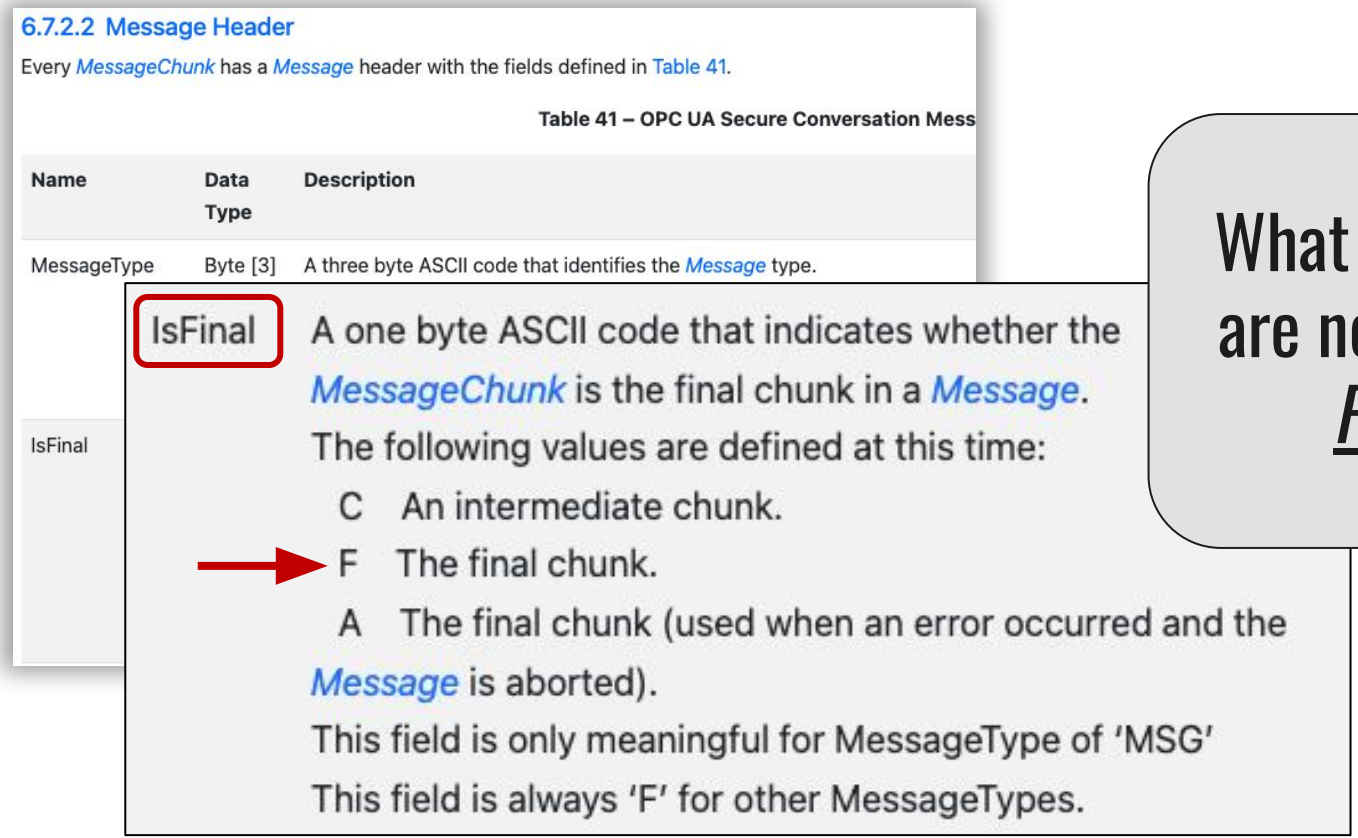
Every *MessageChunk* has a *Message* header with the fields defined in Table 41.

Table 41 – OPC UA Secure Conversation Mess

Name	Data Type	Description
MessageType	Byte [3]	A three byte ASCII code that identifies the *Message* type.
IsFinal		

IsFinal	A one byte ASCII code that indicates whether the *MessageChunk* is the final chunk in a *Message*. The following values are defined at this time: C An intermediate chunk. F The final chunk. A The final chunk (used when an error occurred and the *Message* is aborted). This field is only meaningful for MessageType of 'MSG' This field is always 'F' for other MessageTypes.

OPC UA Protocol Research: Documentation Deep Dive



6.7.2.2 Message Header

Every *MessageChunk* has a *Message* header with the fields defined in Table 41.

Table 41 – OPC UA Secure Conversation Mess

Name	Data Type	Description
MessageType	Byte [3]	A three byte ASCII code that identifies the *Message* type.
IsFinal		

IsFinal

A one byte ASCII code that indicates whether the *MessageChunk* is the final chunk in a *Message*.
The following values are defined at this time:

C An intermediate chunk.

F The final chunk.

A The final chunk (used when an error occurred and the *Message* is aborted).

This field is only meaningful for MessageType of 'MSG'
This field is always 'F' for other MessageTypes.

What happens if we are not sending the *Final* flag?

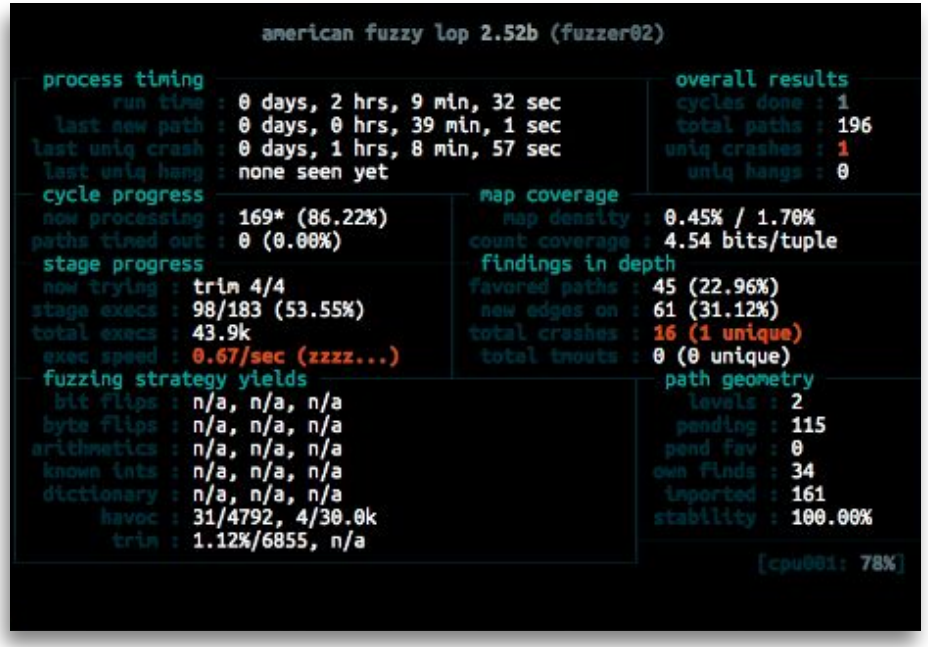
OPC UA Protocol Research: Custom Client

- Built our own OPC-UA protocol stack
- Customized client to our vuln research needs
- Ability to control every aspect of the message, to the bit level

```
opcua = OPCUA(ip_addr, port, query_string)
opcua.session_timeout = 3600 * 1000 # 1hr
opcua.requested_lifetime = 3600 * 1000 # 1hr
opcua.max_chunk_size = max_chunk_size
opcua.create_session()
```

OPC UA Protocol Research: Coverage Fuzzing

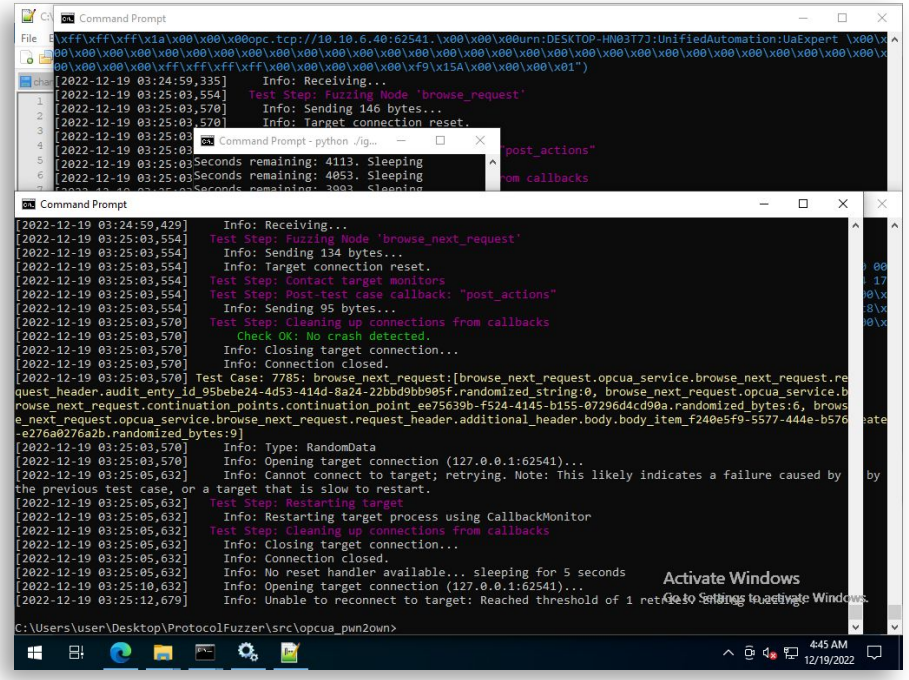
- Used AFL, WinAFL, LibFuzzer
- Four targets fuzzed
- Found 5 memory corruptions



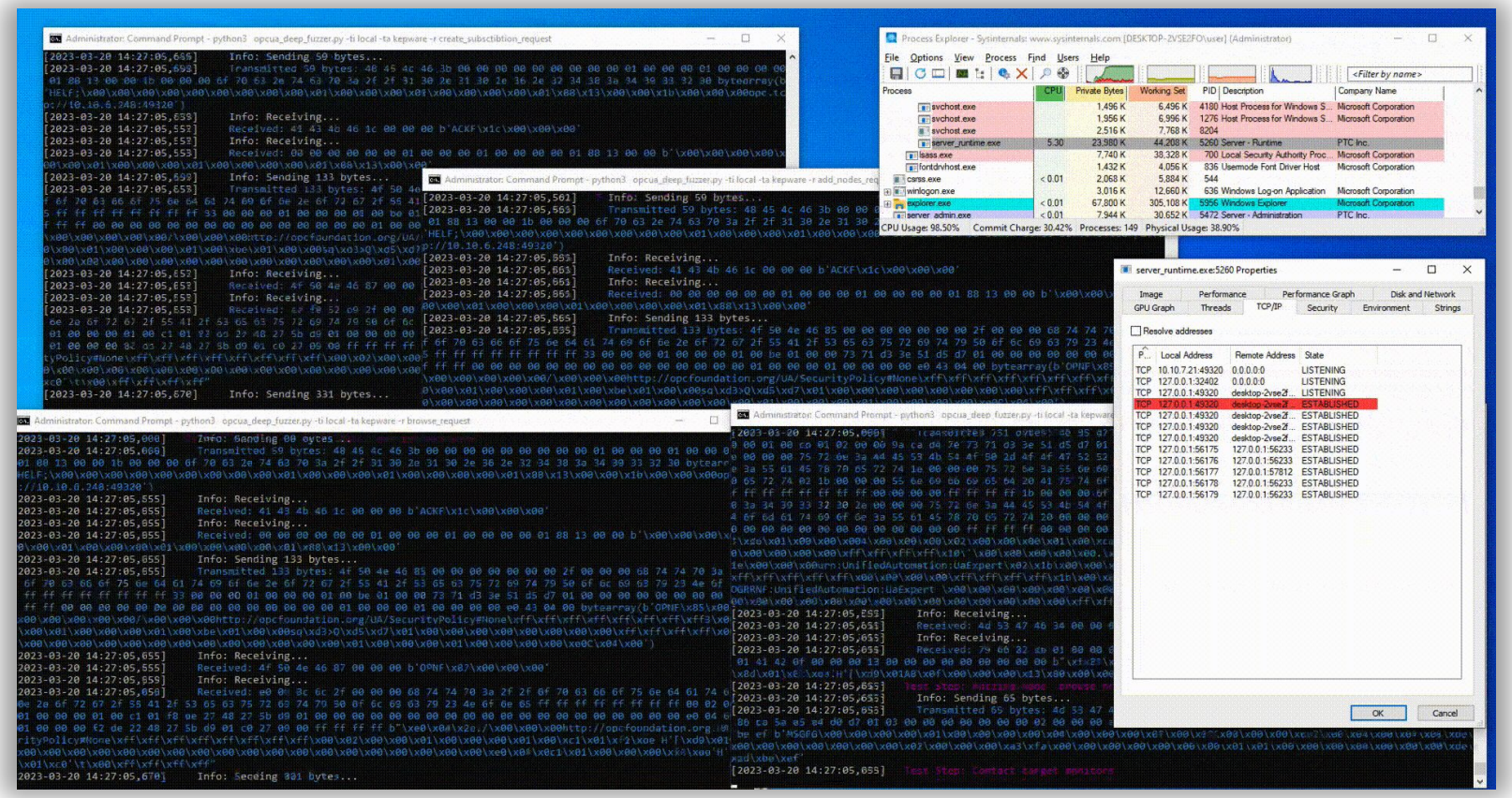
OPC UA Protocol Research: Network Fuzzing

- Based on boofuzz fuzzer
- Fuzzing 6 OPC UA services
- Found two memory corruptions

Released tool as open source!



OPC UA Protocol Research: Network Fuzzing



CLAROTY

Denial of Service – Attack Concepts

Resource exhaustion - uncontrolled memory management

- Chunk Flooding
- Unlimited ConditionRefresh Attack
- Unlimited Persistent Monitored Subscriptions
- Unlimited Open Channels

Threads deadlock

- Worker Starvation

Use-after-free bugs

- Method Calling From Dead Session
- Add/Remove From Namespace While Browsing

Buffer overflows - heap/stack corruption

- Unicode Conversion - OOB Write

Uncaught exceptions

- Parser Bug - Dissecting Malformed OPC-UA Data Type

Busy loops / unlimited recursions – call-stack overflow

- Complex Deep Nested Variants (OTORIO)
- Certificate Chain Loop (Sector7)
- Unlimited Translate Browse Path (JFrog)

Example

OPC UA DoS CVE-2022-29864

Denial of Service – Attack Concepts

Resource exhaustion - uncontrolled memory management

- **Chunk Flooding**
- Unlimited ConditionRefresh Attack
- Unlimited Persistent Monitored Subscriptions
- Unlimited Open Channels

Threads deadlock

- Worker Starvation

Use-after-free bugs

- Method Calling From Dead Session
- Add/Remove From Namespace While Browsing

Buffer overflows - heap/stack corruption

- Unicode Conversion - OOB Write

Uncaught exceptions

- Parser Bug - Dissecting Malformed OPC-UA Data Type

Busy loops / unlimited recursions – call-stack overflow

- Complex Deep Nested Variants (OTORIO)
- Certificate Chain Loop (Sector7)
- Unlimited Translate Browse Path (JFrog)

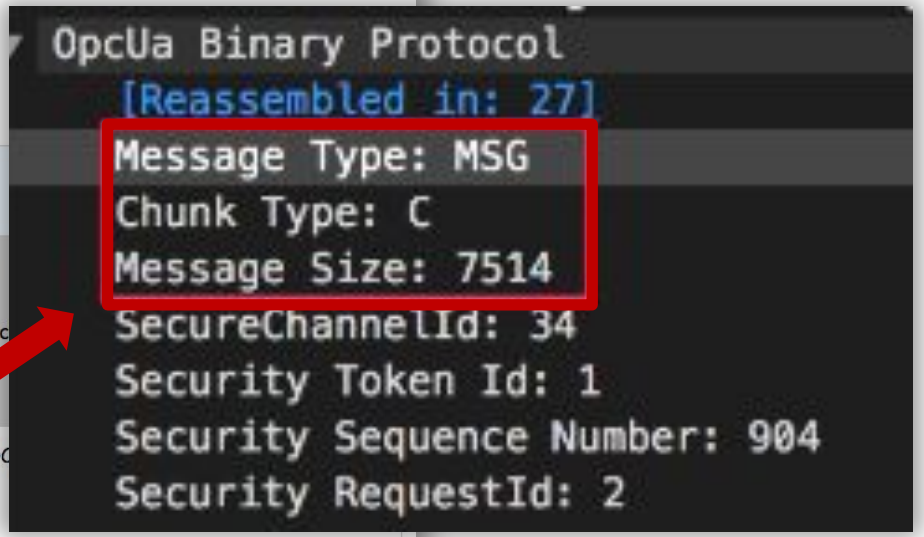
Denial of Service - Chunk Flooding

6.7.2.2 Message Header ↑

Every *MessageChunk* has a *Message* header with the fields defined in Table 41.

Table 41 – OPC UA Secure Conversation Message header

Name	Data Type	Description
MessageType	Byte [3]	A three byte ASCII code that identifies the *Message* type. The following values are defined at this time: MSG A *Message* secured with the keys associated with a c OPN OpenSecureChannel *Message*. CLO CloseSecureChannel *Message*.
IsFinal	Byte	A one byte ASCII code that indicates wh the *MessageC* The following values are defined at time: C An intermediate chunk. F The final chunk. A The final chunk (used when an error occurred and the *Message* is aborted). This field is only meaningful for MessageType of 'MSG' This field is always 'F' for other MessageTypes.



CLAROTY

Denial of Service - Chunk Flooding

```
private bool ProcessRequestMessage(uint messageType, ArraySegment<byt
{
        ...
        ...

    try
    {
        // check for an abort.
        if (TcpMessageType.IsAbort(messageType))
        {
            Utils.Trace("Request was aborted.");
            chunksToProcess = GetSavedChunks(requestId, messageBody);
            return true;
        }

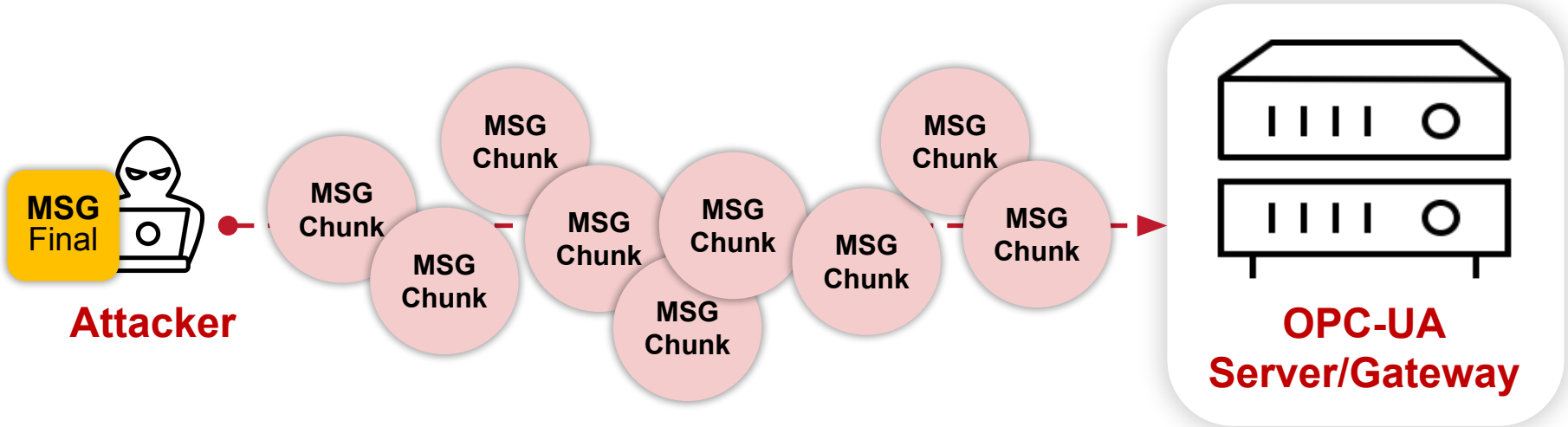
        // check if it is necessary to wait for more chunks.
        if (!TcpMessageType.IsFinal(messageType))
        {
            SaveIntermediateChunk(requestId, messageBody);
            return true;
        }
```

OPC-UA .NET Stack

CLAROTY

Denial of Service - Chunk Flooding

```
while !isFinalChunk:
    add(chunk)
```



Denial of Service - Chunk Flooding

```
while !isFinalChunk:
    add(chunk)
```

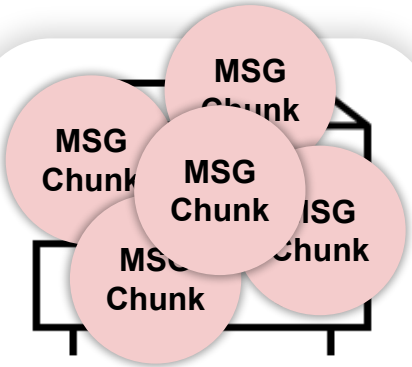


Denial of Service - Chunk Flooding



Attacker



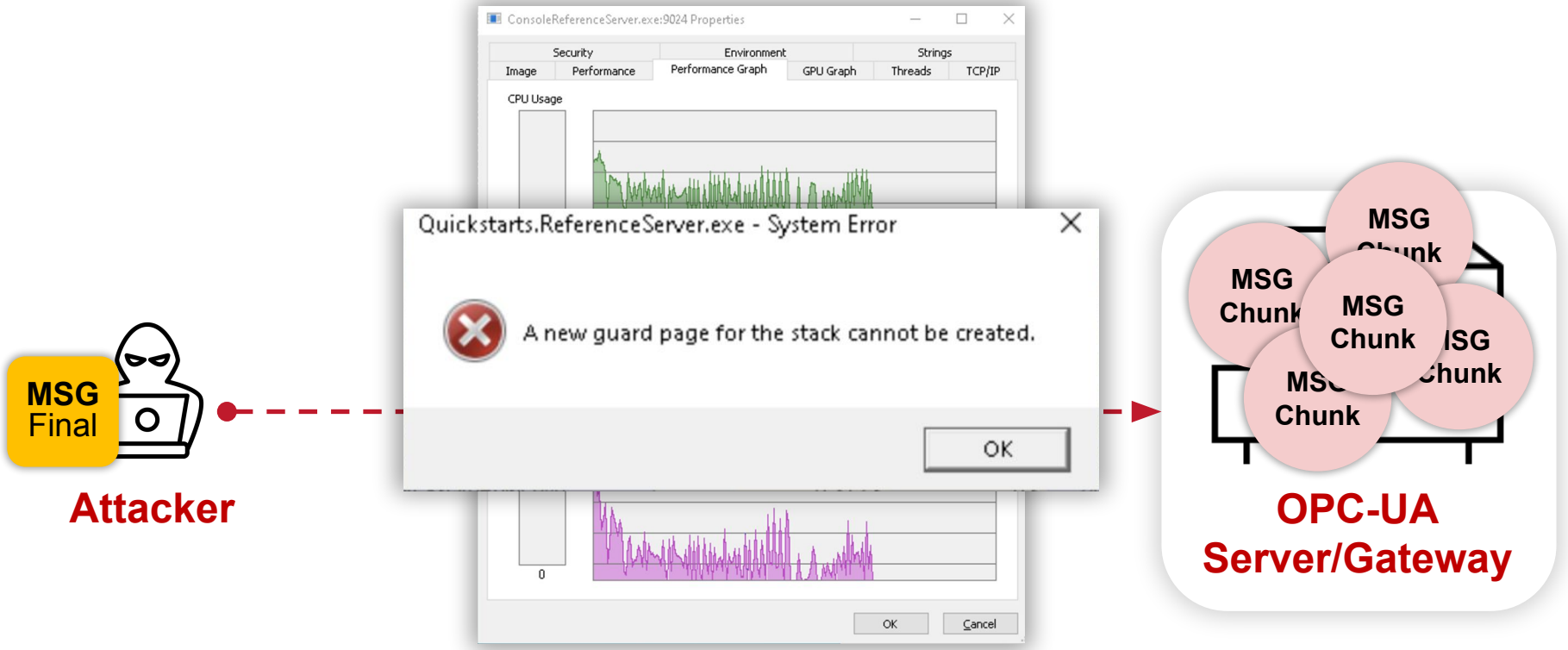


OPC-UA
Server/Gateway

CLAROTY

Denial of Service - Chunk Flooding

CVE-2022-21208 CVE-2022-24381
CVE-2022-25761 CVE-2022-25888
CVE-2022-25304 **CVE-2022-29864**

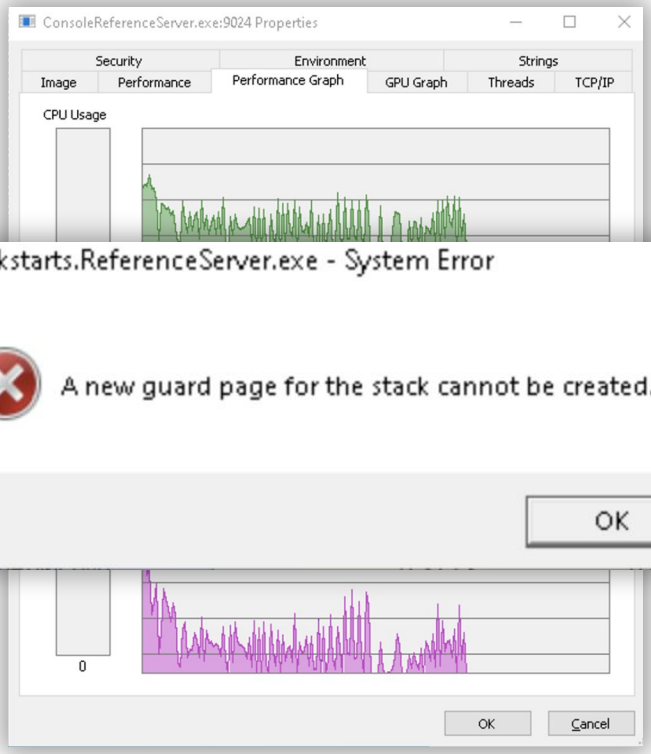


CLAROTY

Denial of Service - Chunk Flooding

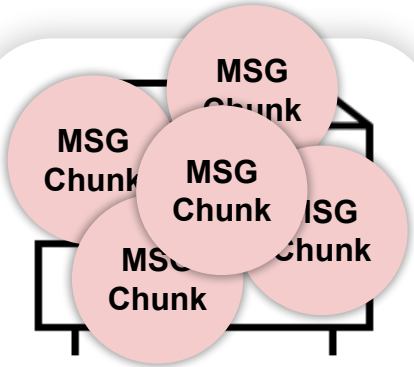


CVE-2022-21208 CVE-2022-24381
CVE-2022-25761 CVE-2022-25888
CVE-2022-25304 **CVE-2022-29864**





Attacker



OPC-UA Server/Gateway

CLAROTY

Example

OPC UA DoS CVE-2022-1748

Denial of Service – Attack Concepts

Resource exhaustion - uncontrolled memory management

- Chunk Flooding
- Unlimited ConditionRefresh Attack
- Unlimited Persistent Monitored Subscriptions
- Unlimited Open Channels

Threads deadlock

- Worker Starvation

Use-after-free bugs

- **Method Calling From Dead Session**
- Add/Remove From Namespace While Browsing

Buffer overflows - heap/stack corruption

- Unicode Conversion - OOB Write

Uncaught exceptions

- Parser Bug - Dissecting Malformed OPC-UA Data Type

Busy loops / unlimited recursions – call-stack overflow

- Complex Deep Nested Variants (OTORIO)
- Certificate Chain Loop (Sector7)
- Unlimited Translate Browse Path (JFrog)

Denial of Service - Method Calling From Dead Session

5 Service Sets ↑ ⊖ ⊕

5.11 Method Service Set ↑ ⊖ ⊕

5.11.2 Call ↑ ⊖ ⊕

5.11.2.1 Description ↑

```
@uamethod
def multiply(parent, x, y):
    print("multiply method call with parameters: ", x, y)
    return x * y
```

Example to exposed function (python-opcua)

This *Service* is used to call (invoke) a list of *Methods*.

This *Service* provides for passing input and output arguments to/from a *Method*. These arguments are defined by *Properties* of the *Method*.

If the *Method* is invoked in the context of a *Session* and the *Session* is terminated, the results of the *Method's* execution cannot be returned to the *Client* and are discarded. This is independent of the task actually performed at the *Server*.

The order the operations are processed in the *Server* is not defined and depends on the different tasks and the internal *Server* logic. If a *Method* is contained in more than one operation, the order of the processing is undefined. If a *Client* requires sequential processing the *Client* needs separate *Service* calls.

https://reference.opcfoundation.org/v104/Core/docs/Part4/5.11.2/

Denial of Service - Method Calling From Dead Session

5 Service Sets ↑ ⊖ ⊕

5.11 Method Service Set ↑ ⊖ ⊕

5.11.2 Call ↑ ⊖ ⊕

5.11.2.1 Description ↑

```
@uamethod
def multiply(parent, x, y):
    print("multiply method call with parameters: ", x, y)
    return x * y
```

Example to exposed function (python-opcua)

This *Service* is used to call (invoke) a list of *Methods*.

This *Service* provides for passing input and output arguments to/from a *Method*. These arguments are defined by *Properties* of the *Method*.

If the *Method* is invoked in the context of a *Session* and the *Session* is terminated, the results of the *Method's* execution cannot be returned to the *Client* and are discarded. This is independent of the task actually performed at the *Server*.

The order the operations are processed in the *Server* is not defined and depends on the different tasks and the internal *Server* logic. If a *Method* is contained in more than one operation, the order of the processing is undefined. If a *Client* requires sequential processing the *Client* needs separate *Service* calls.

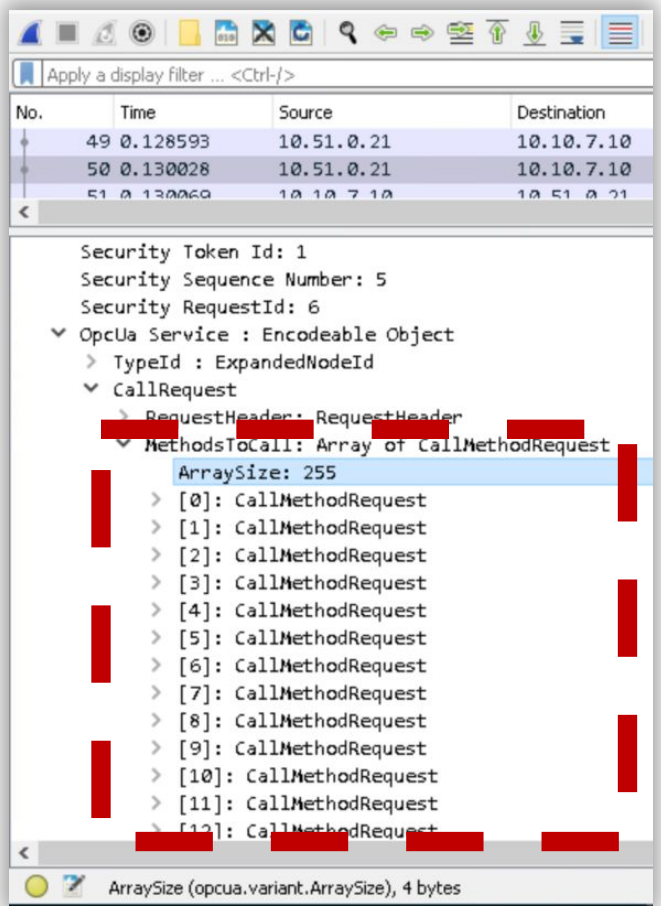
https://reference.opcfoundation.org/v104/Core/docs/Part4/5.11.2/

Denial of Service - Method Calling From Dead Session

Did all stacks implement this correctly?

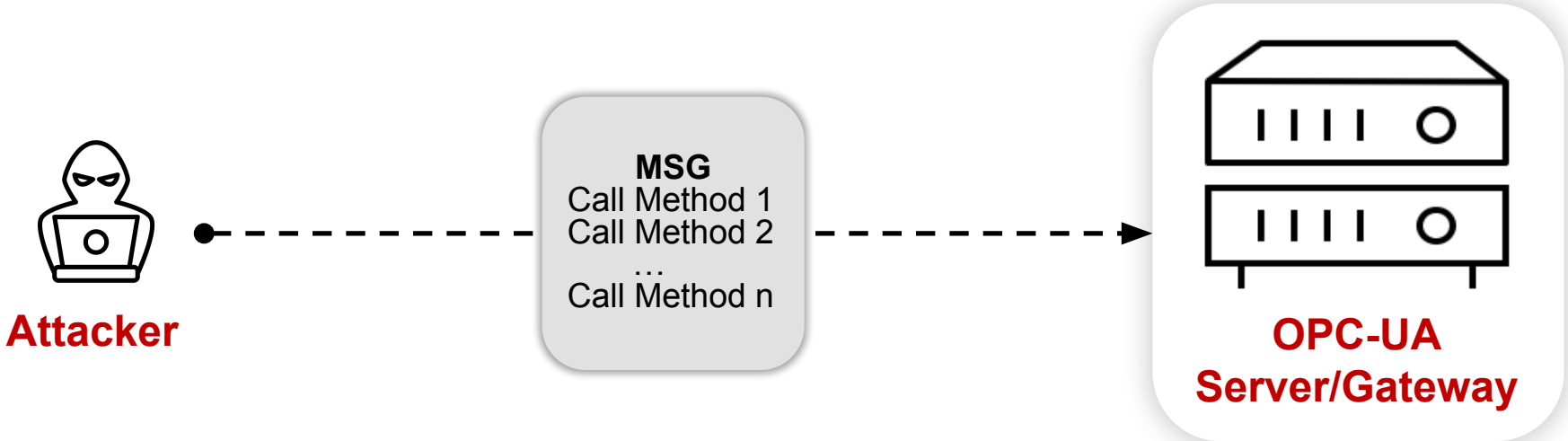
Exploit:

- Sending many Call Method Request
- And immediately close the session

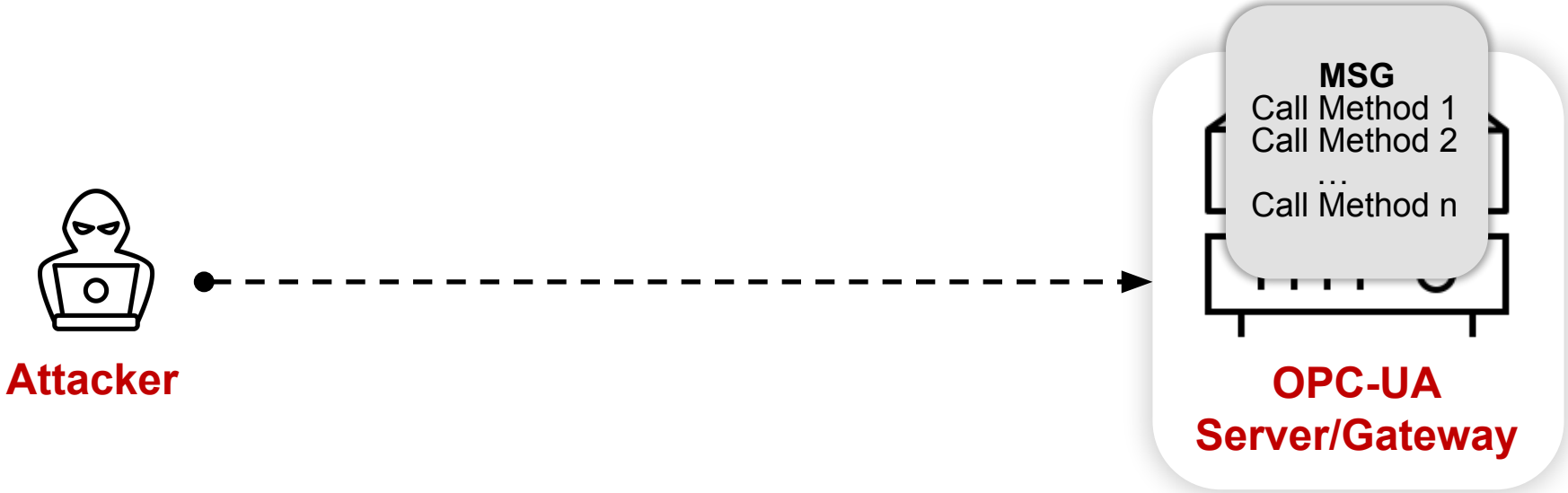


CLAROTY

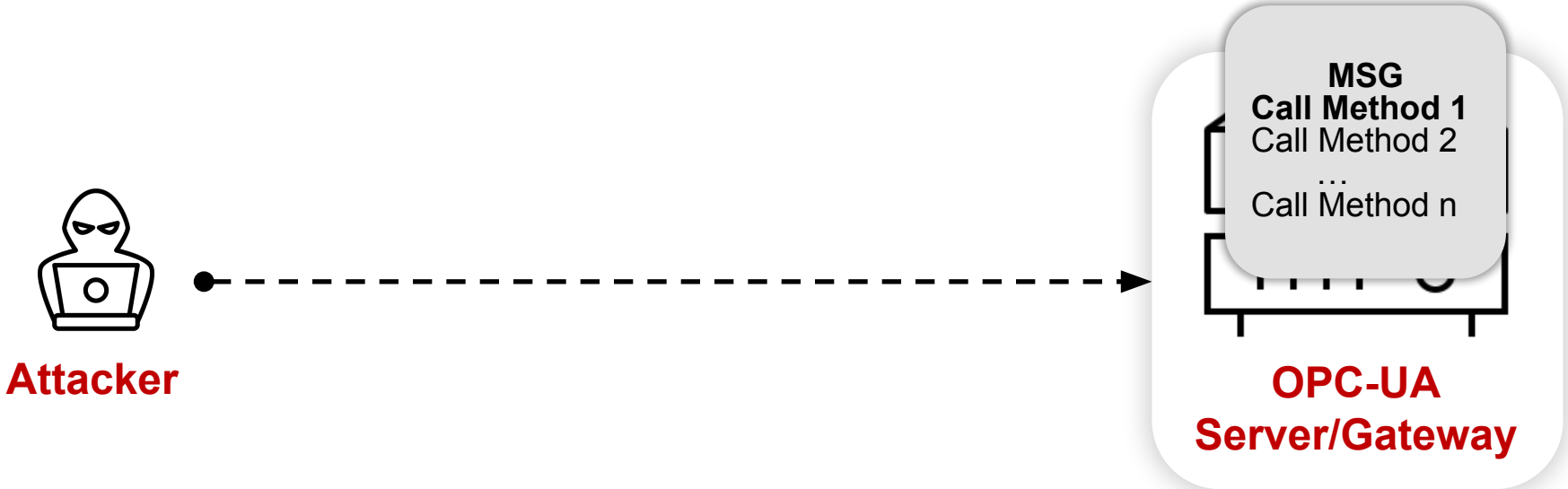
Denial of Service - Method Calling From Dead Session



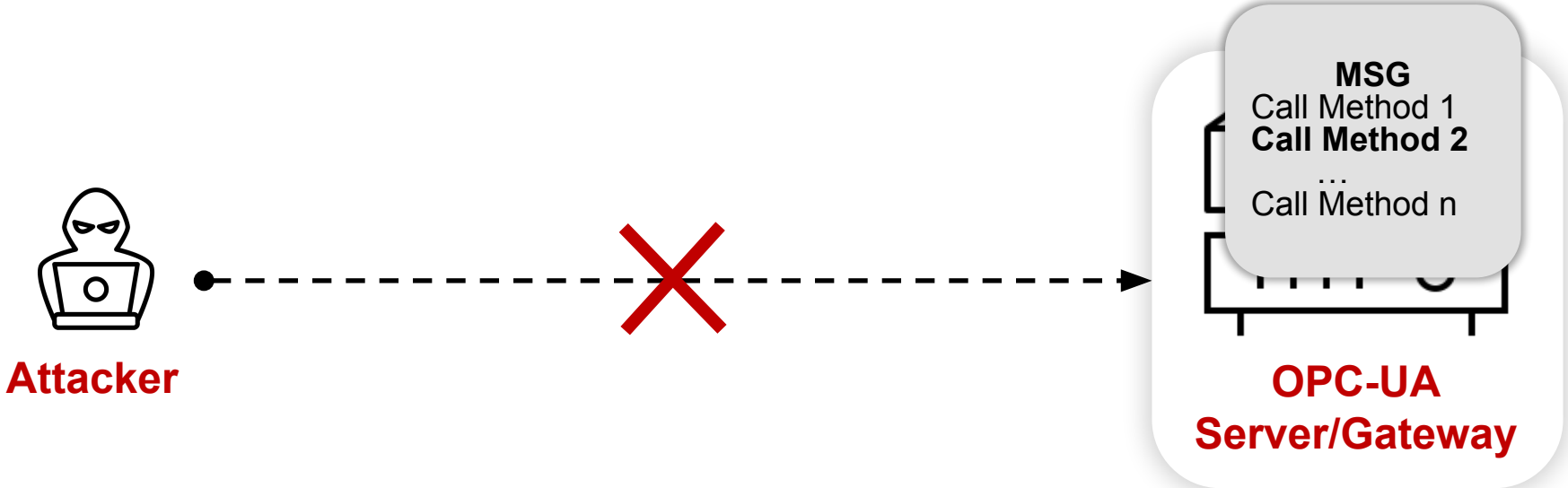
Denial of Service - Method Calling From Dead Session



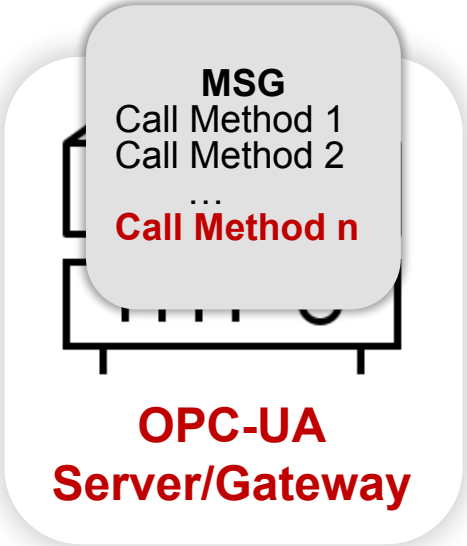
Denial of Service - Method Calling From Dead Session



Denial of Service - Method Calling From Dead Session



Denial of Service - Method Calling From Dead Session



Denial of Service - Method Calling From Dead Session

MSG
Call Method 1
Call Method 2
…
Call Method n

Softing Secure Integration Server

```
OTServerMethodCallRequest::setInputReturns_API((int)a3, 1, &src);
OTServerDataTransaction::getSession(
  (OTServerDataTransaction *)&OTServerDataTransaction,
  OTServerMethodCallTransaction);
```

CVE-2022-1748

Denial of Service - Method Calling From Dead Session

Softing Secure Integration Server

MSG
Call Method 1
Call Method 2
…
Call Method n

```
OTServerMethodCallRequest::setInputReturns_API((int)a3, 1, &src);
OTServerDataTransaction::getSession(
  (OTServerDataTransaction *)&OTServerDataTransaction,
  OTServerMethodCallTransaction);
```

```
(d24.1740): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
ntdll!RtlEnterCriticalSection+0xd:
00007ff9`1529faad f00fba710800    lock btr dword ptr [rcx+8],0 ds:00000000`00000028=????????
```

CLAROTY

Denial of Service - Method Calling From Dead Session

CVE-2022-1748



Softing Secure Integration Server

MSG
Call Method 1
Call Method 2
…
Call Method n

```
OTServerMethodCallRequest::setInputReturns_API((int)a3, 1, &src);
OTServerDataTransaction::getSession(
  (OTServerDataTransaction *)&OTServerDataTransaction,
  OTServerMethodCallTransaction);
```

```
(d24.1740): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
ntdll!RtlEnterCriticalSection+0xd:
00007ff9`1529faad f00fba710800    lock btr dword ptr [rcx+8],0 ds:00000000`00000028=????????
```

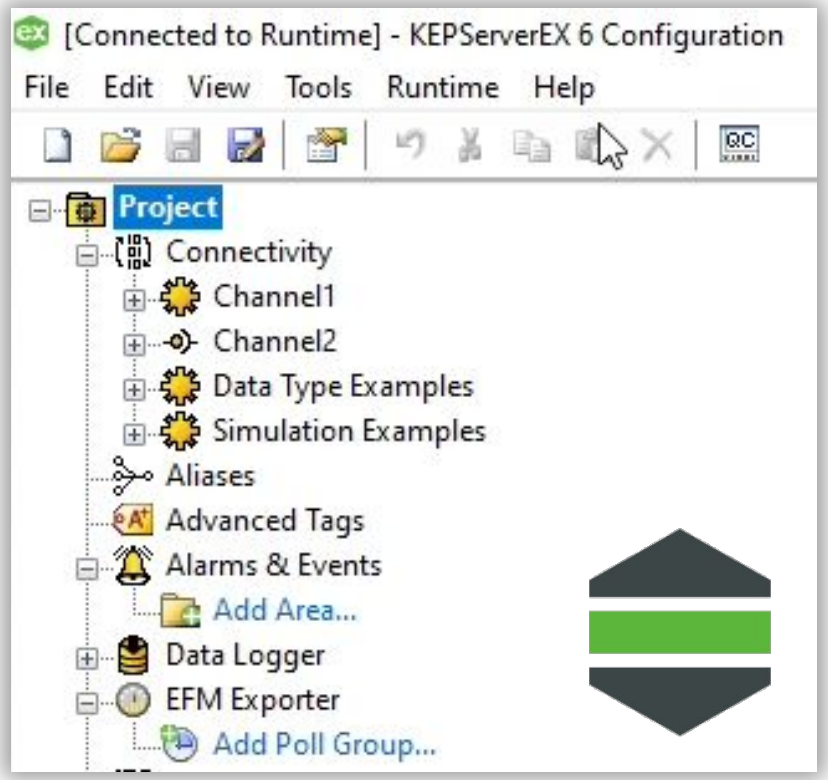
CLAROTY

Example

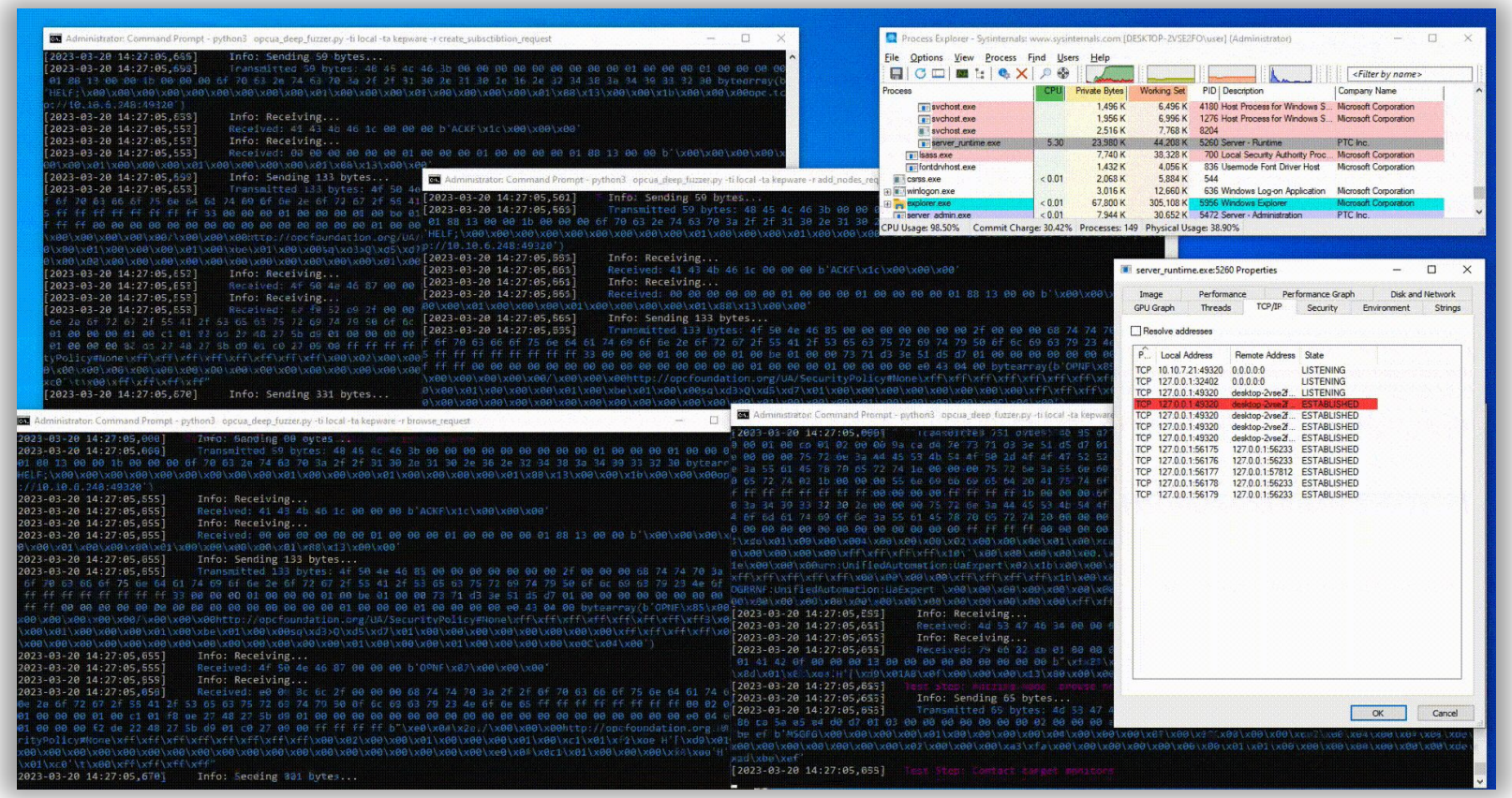
OPC UA RCE CVE-2022-2848, CVE-2022-2825

PTC Kepware KepServerEx

- Industry's leading OPC-UA server, used in biggest manufacturing lines, oil rigs, wind farms, etc.
- Windows-based
- Custom OPC-UA protocol stack
- OPC-UA logic in `server_runtime.exe`
  - 32bit, service (SYSTEM)
  - Customized anti-debugging

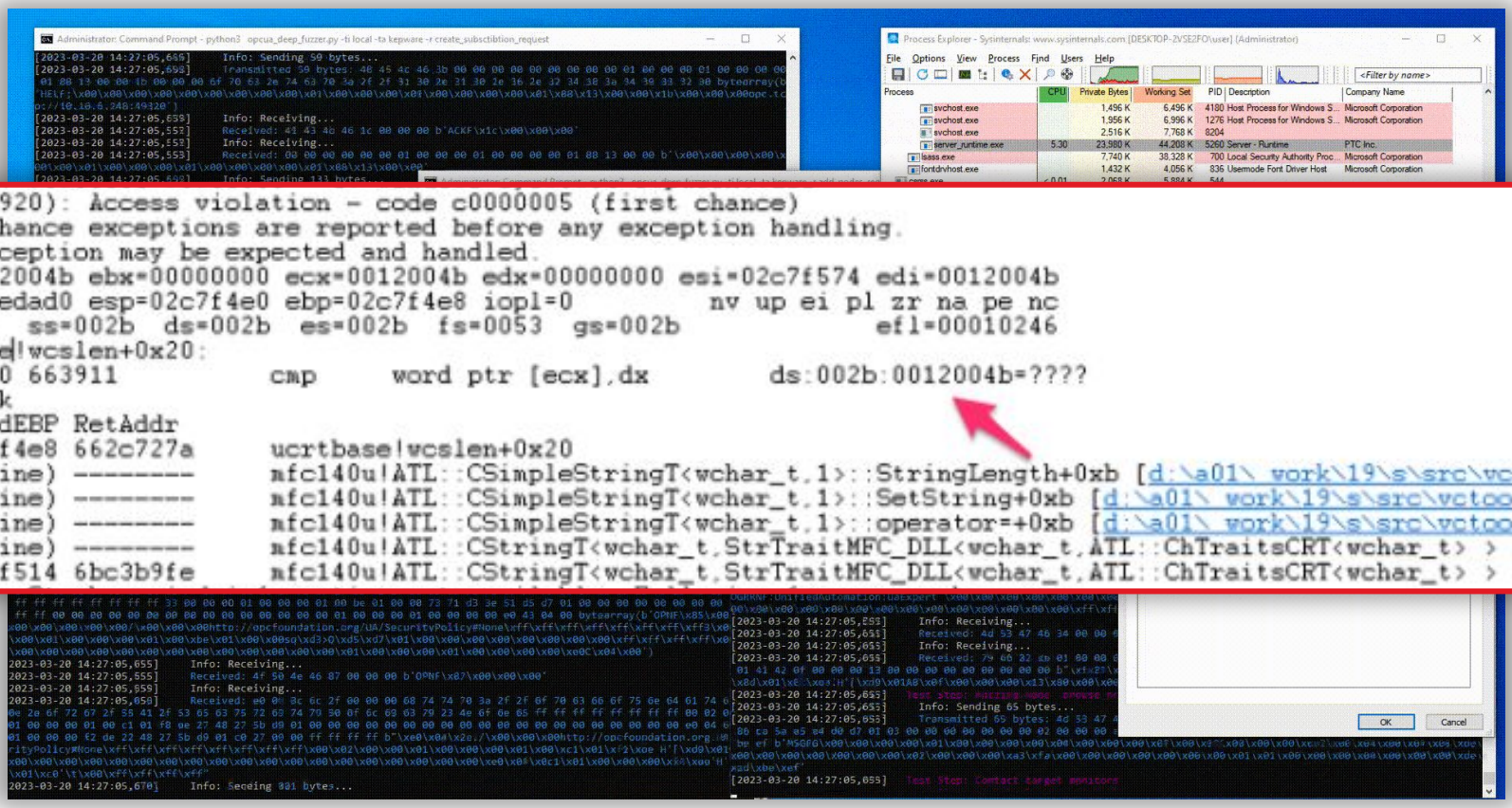


OPC UA Protocol Research: Network Fuzzing



CLAROTY

OPC UA Protocol Research: Network Fuzzing



CLAROTY

KepServerEx Conversion bug - Read/Write Tags

KepServerEx is *trying* to convert UTF-8 to UTF-16

```
1 struct_a1 *__cdecl CUtf8String::ToWide(struct_a1 *struct_for_cstring, char *input_string)
2 {
3   unsigned int number_of_utf16_code_units; // eax
4   struct_a1 *result; // eax
5
6   number_of_utf16_code_units = CUtf8String::GetUtf16Length(input_string);
7   struct_for_cstring->number_of_utf16_code_units = 0;
8   struct_for_cstring->lenght = 7;
9   LOWORD(struct_for_cstring->pointer_to_heap_allocated_buffer) = 0;
10  PERFORM_ALLOCATIONS(struct_for_cstring, number_of_utf16_code_units, 0);
11  result = struct_for_cstring;
12  if ( !struct_for_cstring->number_of_utf16_code_units )
13    return result;
14  if ( struct_for_cstring->lenght >= 8u )
15    result = (struct_a1 *)struct_for_cstring->pointer_to_heap_allocated_buffer;
16  CUtf8String::ToWide(input_string, (char *)result);
17  result = struct_for_cstring;
18  return result;
19 }
```

```
number_of_utf16_code_units = CUtf8String::GetUtf16Length(input_string);
```

CLAROTY

UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3

```
Value: DataValue
  > EncodingMask: 0x01, has value
  Value: Variant
      Variant Type: String (0x0c)
      String:  AAA\xC3
```

Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3



```
41 41 41 c3 00
```

Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3



```
41 41 41 c3 00
```

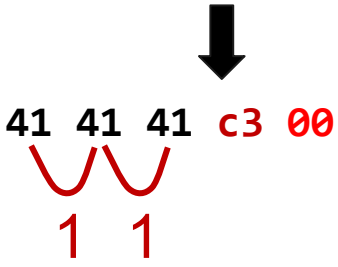
1

Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3

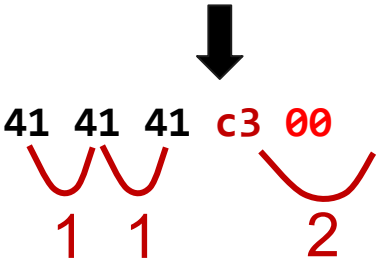


Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3

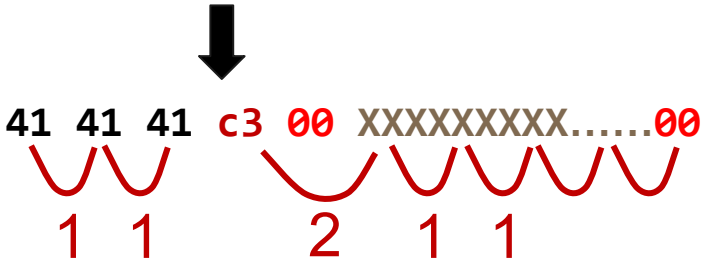


Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3

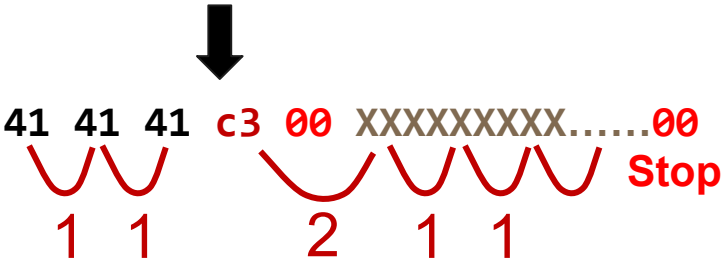


Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA\xC3

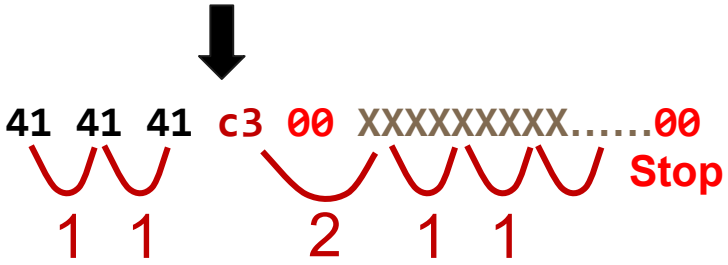


Whatever starts with C3 is probably 2 bytes long



UTF-8 to UTF-16 is Hard

UTF-8: AAA**\xC3**



UTF-16:

\x41\x41\x41\xC3LEAKINGTHEHEAP

CLAROTY

Heap Overflow Primitive

The bug is triggered on both READ_TAG and WRITE_TAG functions

We have heap OOB (read+write)

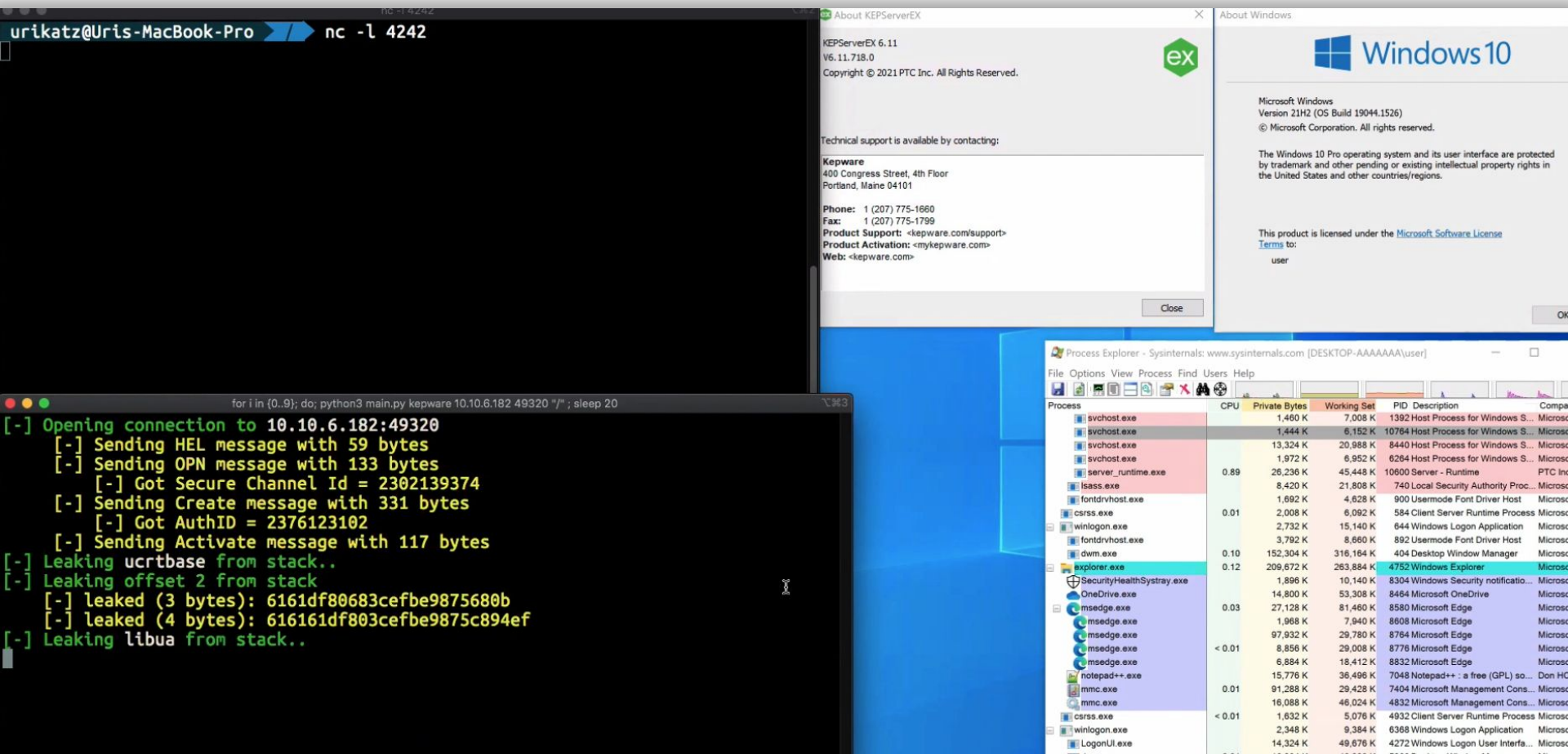
- OOB read → leak pointers to defeat ASLR
- OOB write → construct ROP chain, RCE and PWN

Building the ROP Chain

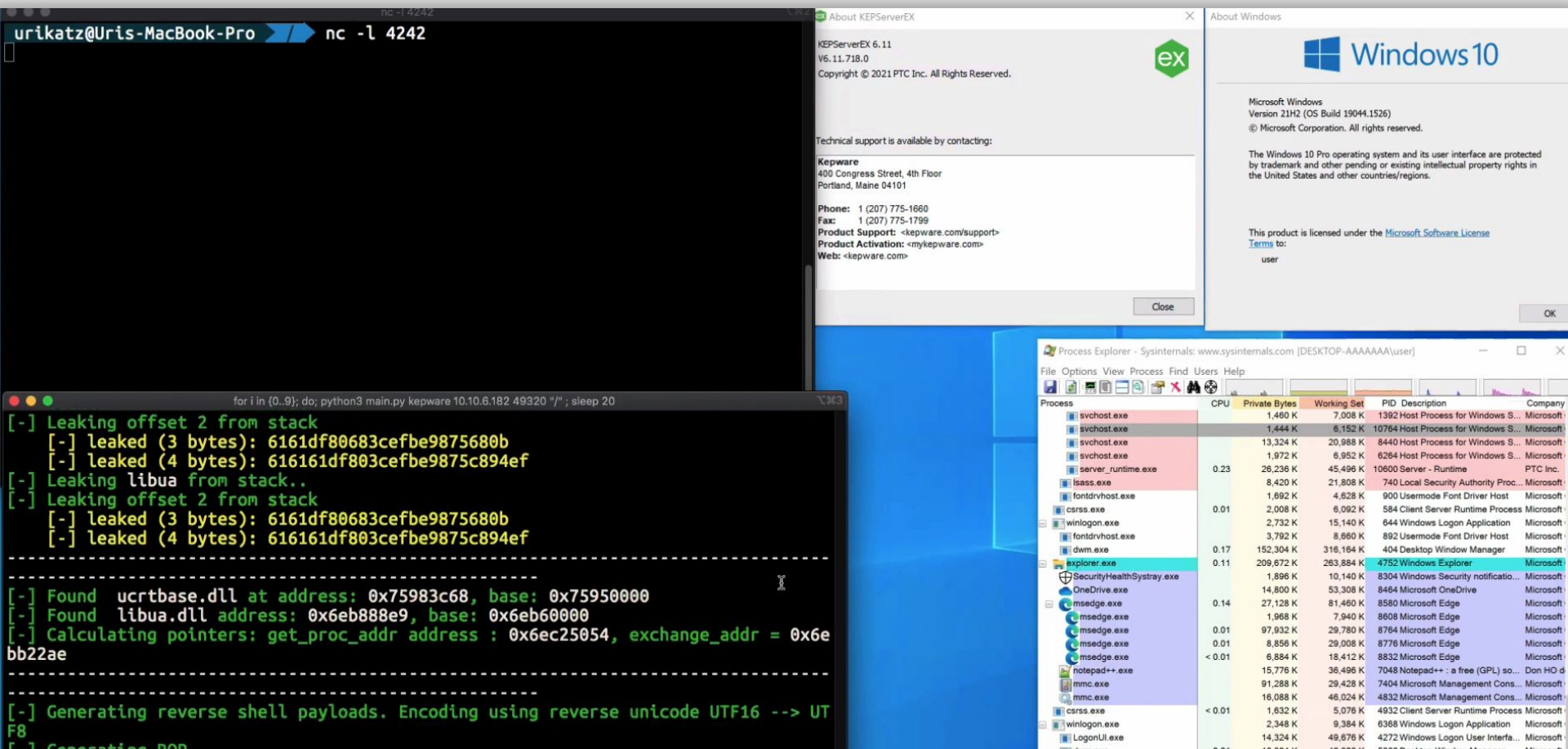
```
if kep_version == KEPWARE_VERSION_OLD:
    eax = get_encoded_address(libua_base + 0x[redacted])
else:
    eax = get_encoded_address(libua_base + 0x[redacted])
edx = get_encoded_address(ucrt_base + 0x[redacted])
edx += get_encoded_address(libua_base + 0x[redacted])
ebx = get_encoded_address(ucrt_base + 0x[redacted])
ebp = get_encoded_address(libua_base + 0x[redacted])
ebp += get_encoded_address(libua_base + 0x[redacted])
edi = get_encoded_address(ucrt_base + 0x[redacted])
edi += get_encoded_address(libua_base + 0x[redacted])
if kep_version == KEPWARE_VERSION_OLD:
    pushad = get_encoded_address(libua_base + 0x[redacted])
else:
    pushad = get_encoded_address(ucrt_base + 0x[redacted])

mspaint_encoded = b'mspaint.exe\x00'.decode("utf-16-le").encode("utf-8")
mspaint_string = b'\xe7\x8d\xad\xe6\x85\xb0\xe6\xb9\xa9\xe2\xb9\xb4\xe7\xa1\xa5e'
```

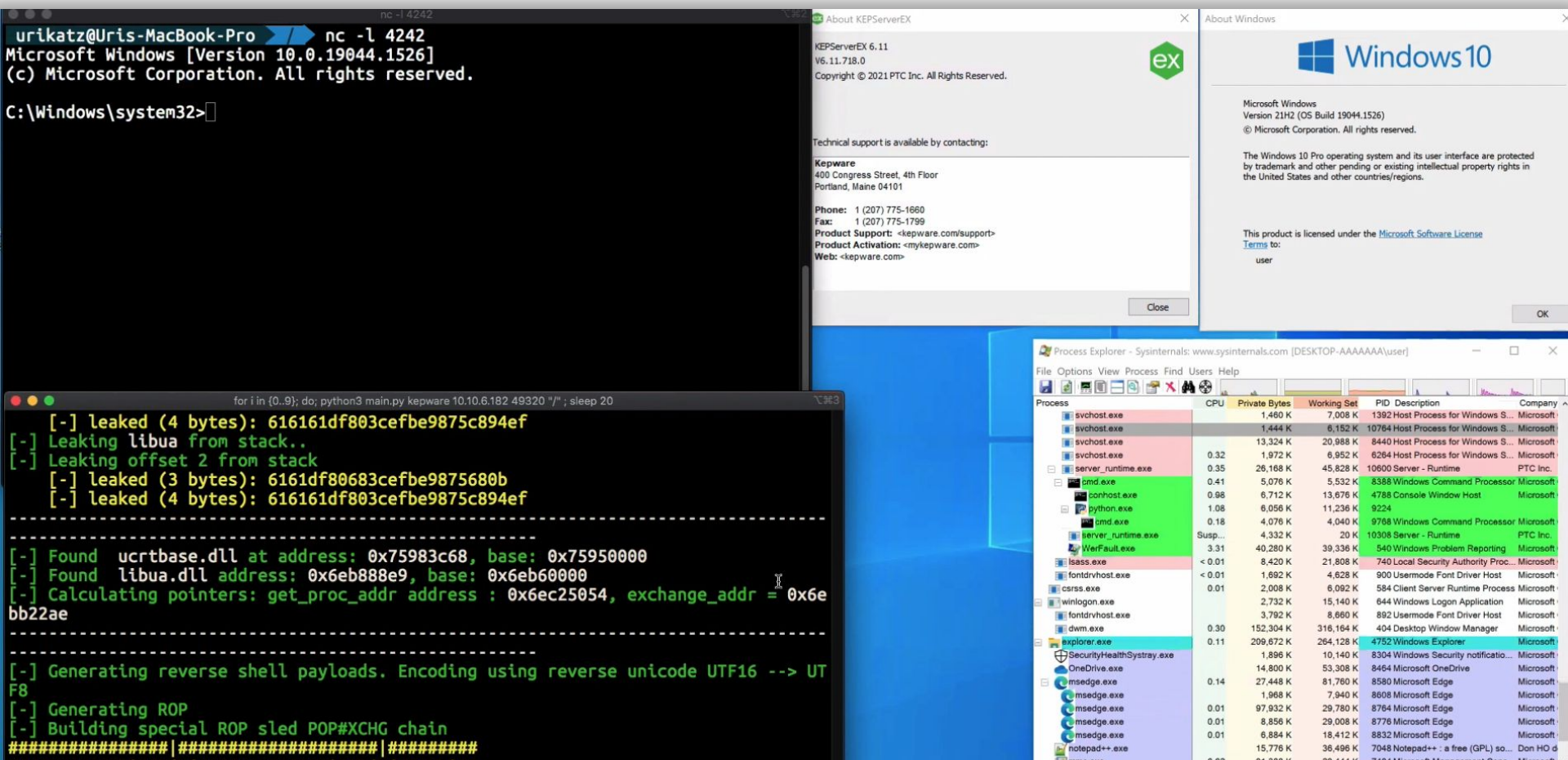
PTC Kepware RCE - Leaking



PTC Kepware RCE - Overwriting Heap



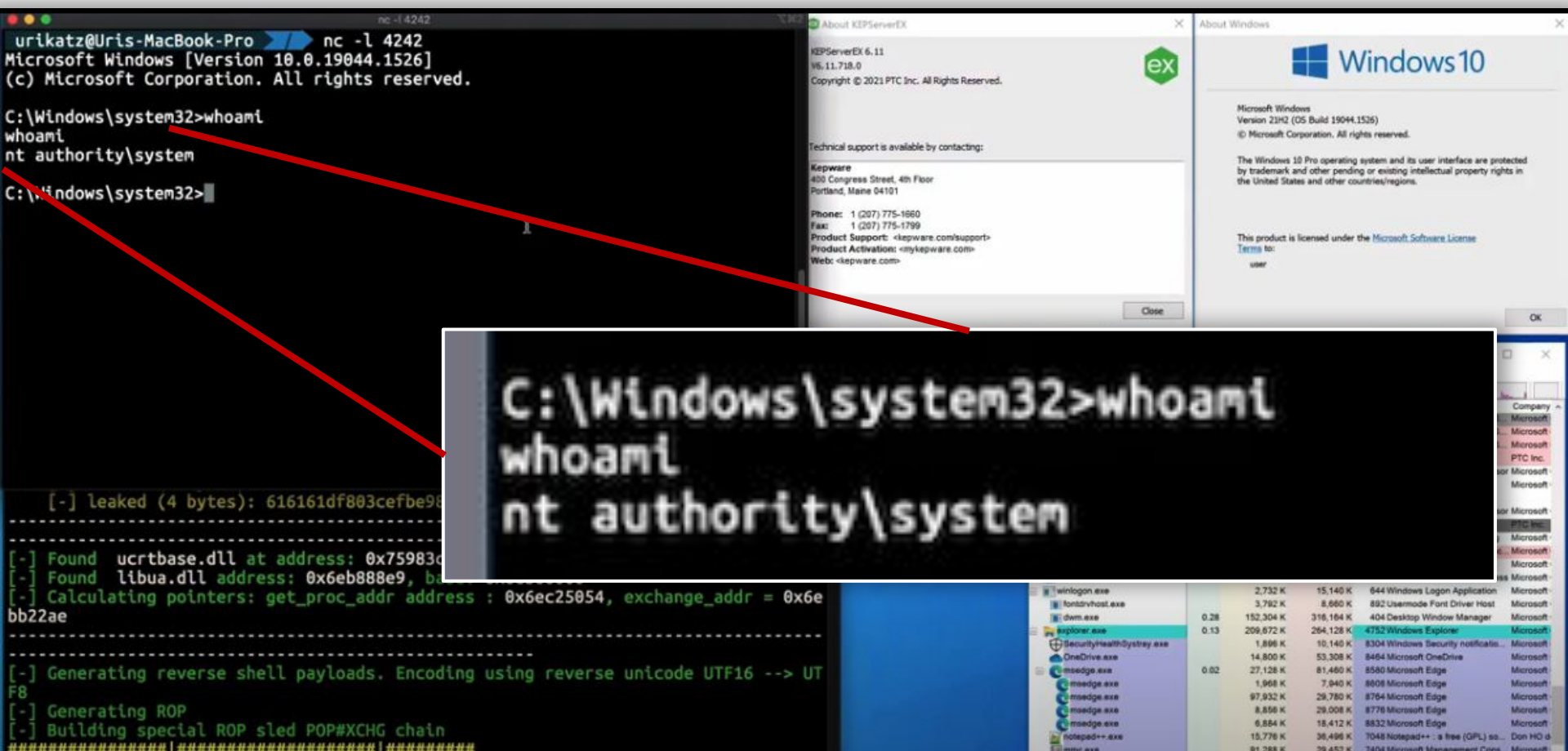
PTC Kepware RCE - Triggering



PTC Kepware RCE

$20,000

CVE-2022-2848
CVE-2022-2825



Summary

OPC UA Research

Pwn2Own OPC UA Attack Techniques & Exploits

Attack Name	CVE	Payload
Chunk Flooding	CVE-2022-29864	DOS
Open Multiple Secure Channels	CVE-2023-32787	DOS
Function Call Null Dereference	CVE-2022-1748	DOS
Malformed UTF8 Conversions	CVE-2022-2825	RCE
Race Change And Browse Address Space	CVE-2023-32172	DOS
Unlimited Condition Refresh	CVE-2023-27321	DOS
Thread Pool Wait Starvation	CVE-2022-30551	DOS

OPC UA Protocol Research: Aftermath

Researched dozens of OPC-UA protocol stacks and products

Found core issues in protocol implementations
~50 CVEs: DoS, Info leaks, RCE
~12 unique generic attacks

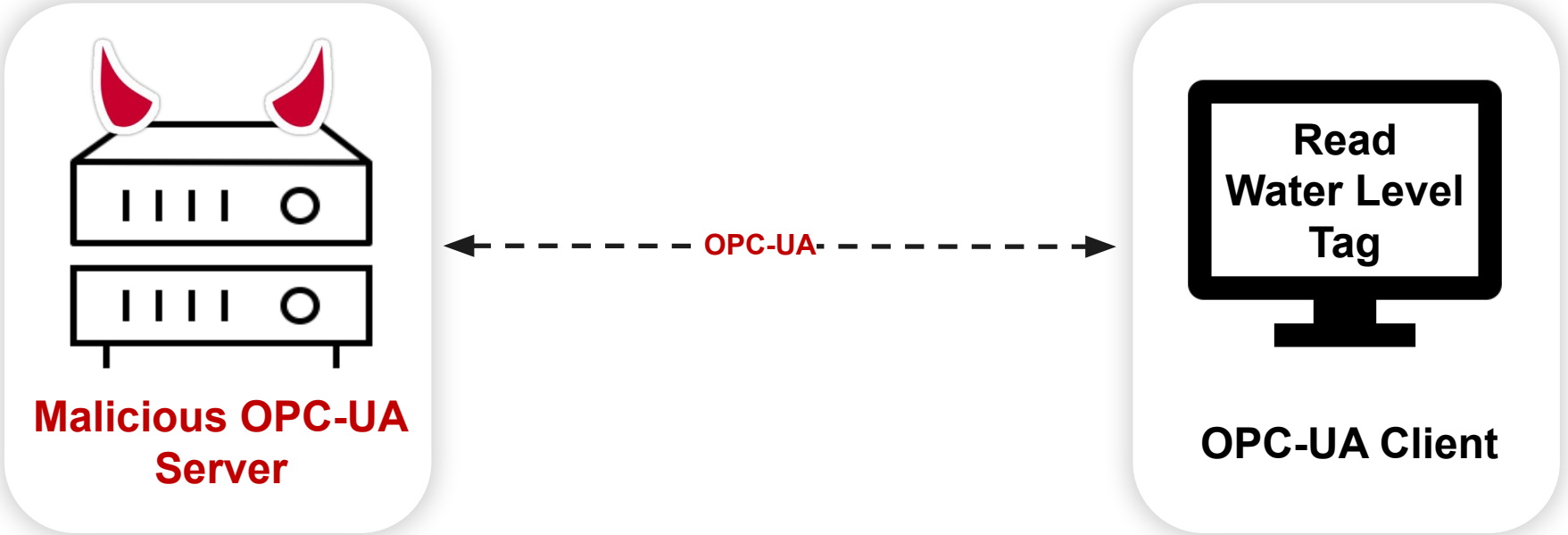
Open-source tools
- OPC-UA fuzzer
- OPC-UA exploitation framework

Three Pwn2Own ICS

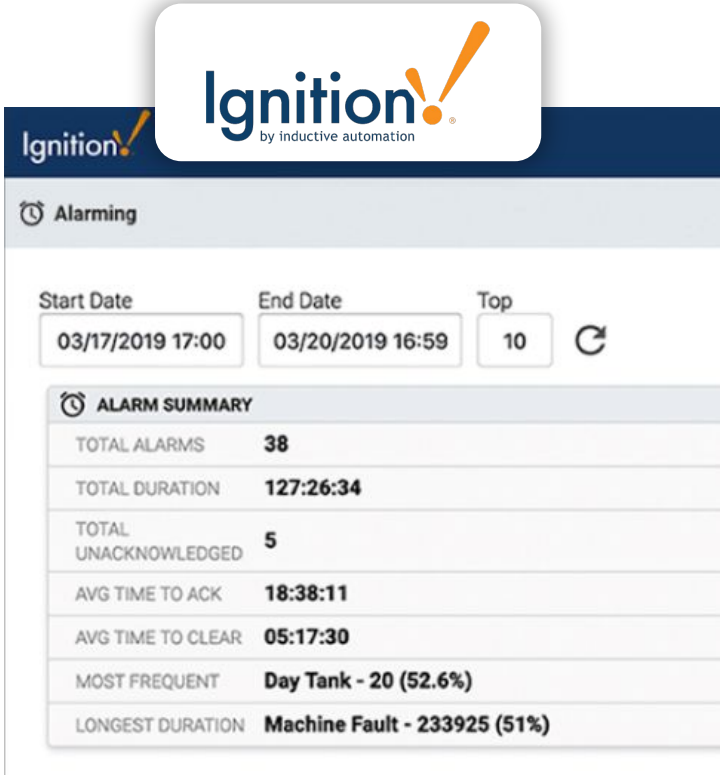
Stack/Application Name	Lang	Complex Deep Nested Variants DoS	Worker Starvation DoS	Long Chunks DoS	Unlimited Monitored Items DoS	U
node-opcua	NodeJS	V	V	CVE-2022-21208	CVE-2022-24375	
open62541	C	V	V	CVE-2022-25761	V	
freeopcua (c++)	C++	V	V	V	CVE-2022-24298	
python-opcua	Python	V	V	CVE-2022-25304	V	
opcua-asyncio	Python	V	V	CVE-2022-25304	V	
eclipse-milo	Java	V	V	V	CVE-2022-25897	
ASNeG OpcUaStack	C++	V	V	CVE-2022-24381	V	
locka99	Rust	CVE-2022-25903	V	CVE-2022-25888	V	
Unified Automation	C++	V	V	V	Fixed, No CVE	
OPC Foundation .NET Stack	C#	CVE-2021-27432 (*	V	CVE-2022-29864	V	
Softing OPC UA SDK	C++	V	V	V	V	
Prosys OPC UA	Java	V	CVE-2022-30551	V	V	
OPC UA Legacy Java Stack	Java	V	CVE-2022-30551	V	V	
Kepware KEPServerEX	C/C++	V	V	V	V	CV CV

Attacking SCADA Client - XSS to RCE

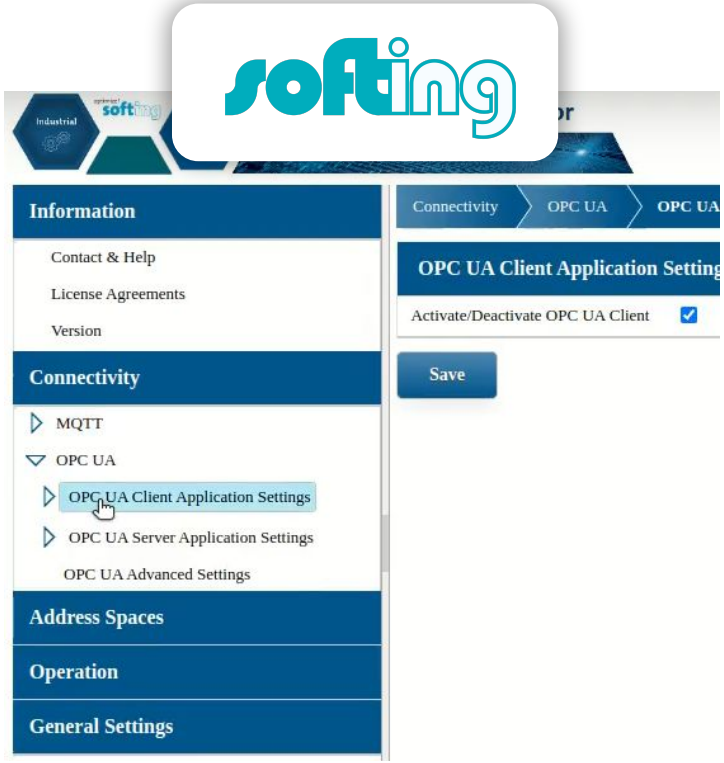
Attacking SCADA clients with OPC-UA support



Web-Based OPC-UA Clients



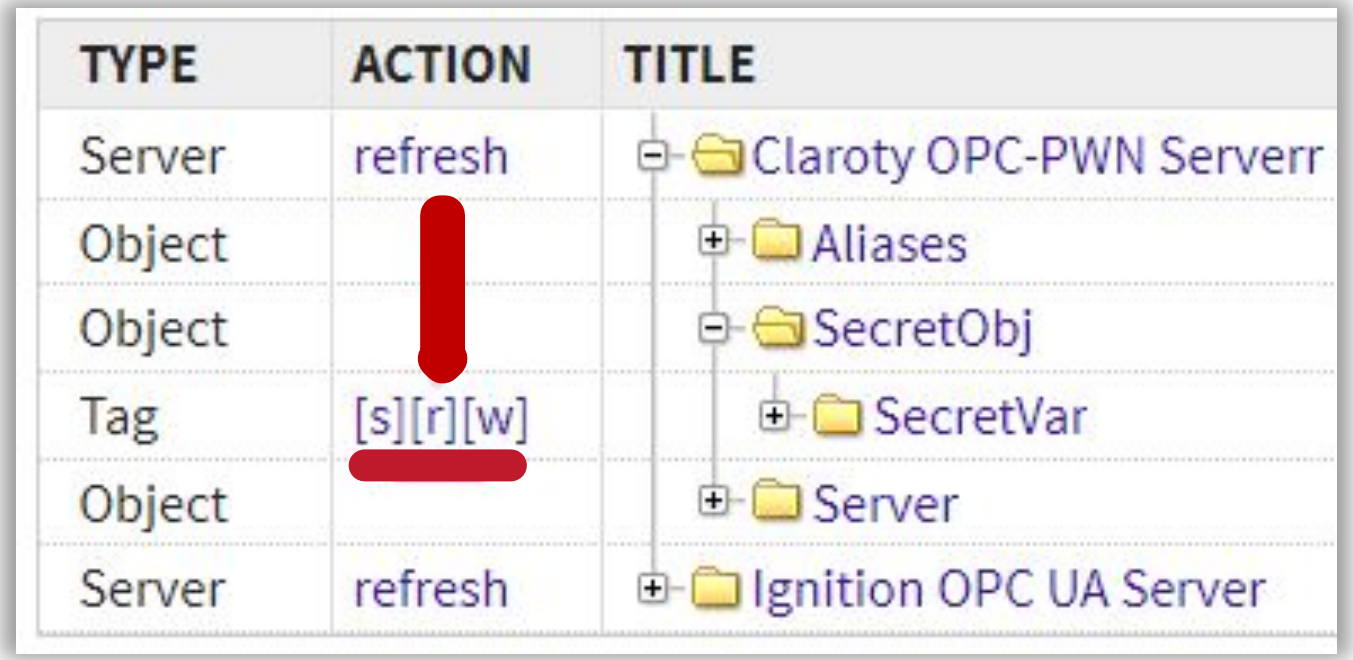
Inductive Automation
Ignition



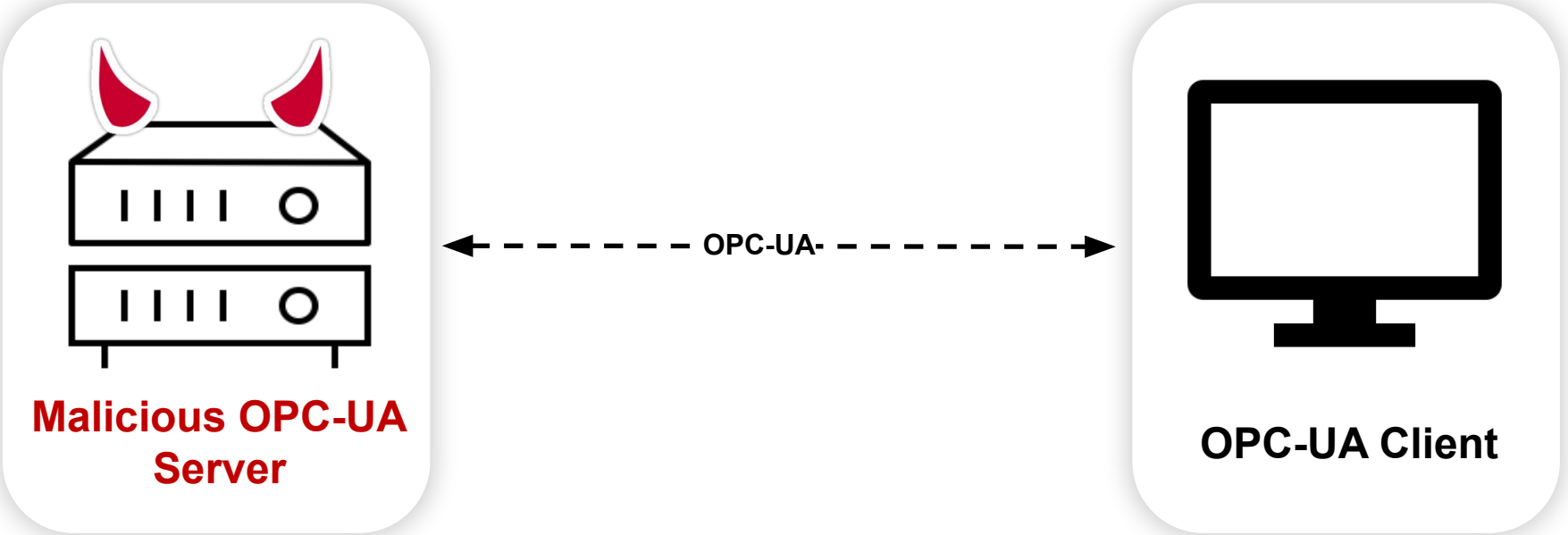
Softing
dataFEED edgeAggregator

CLAROTY

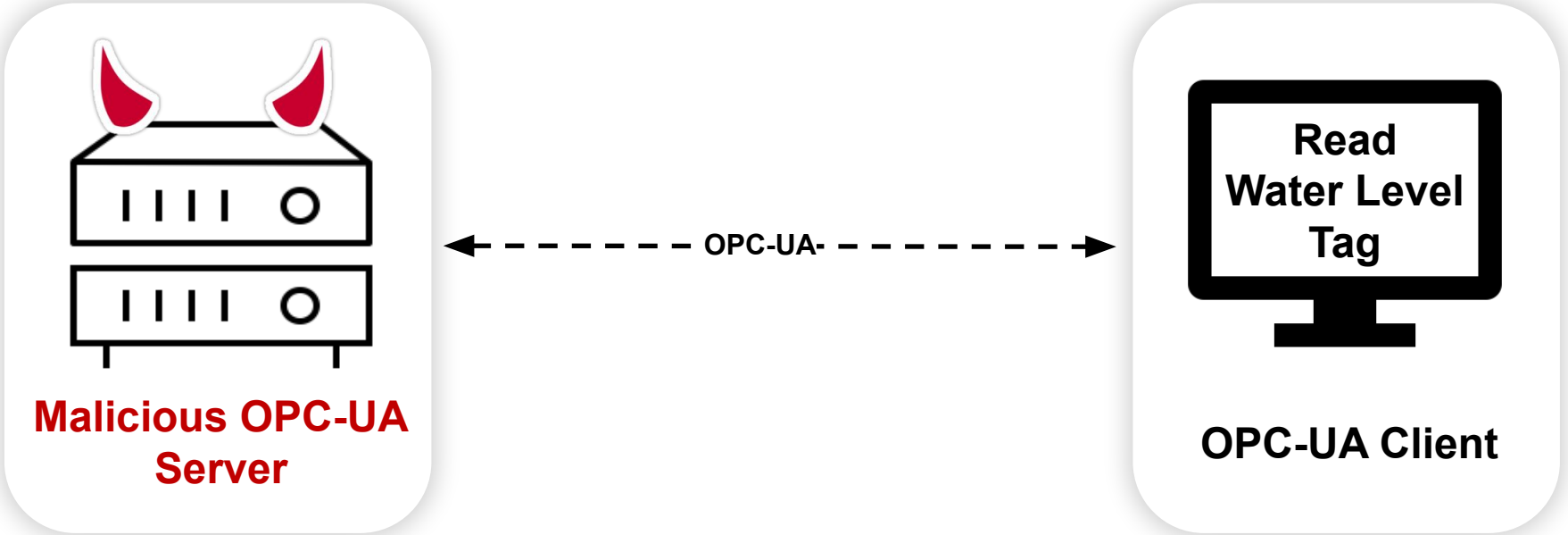
XSS Over OPC-UA



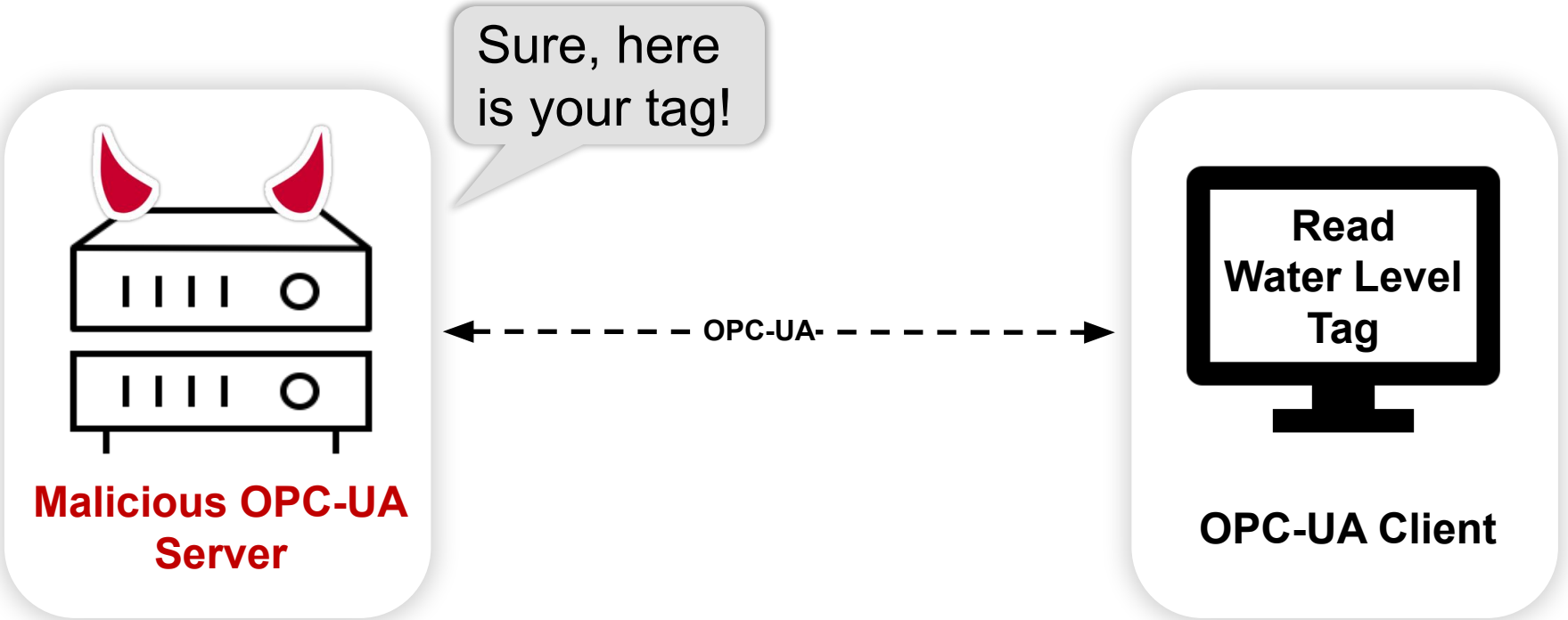
XSS Over OPC-UA



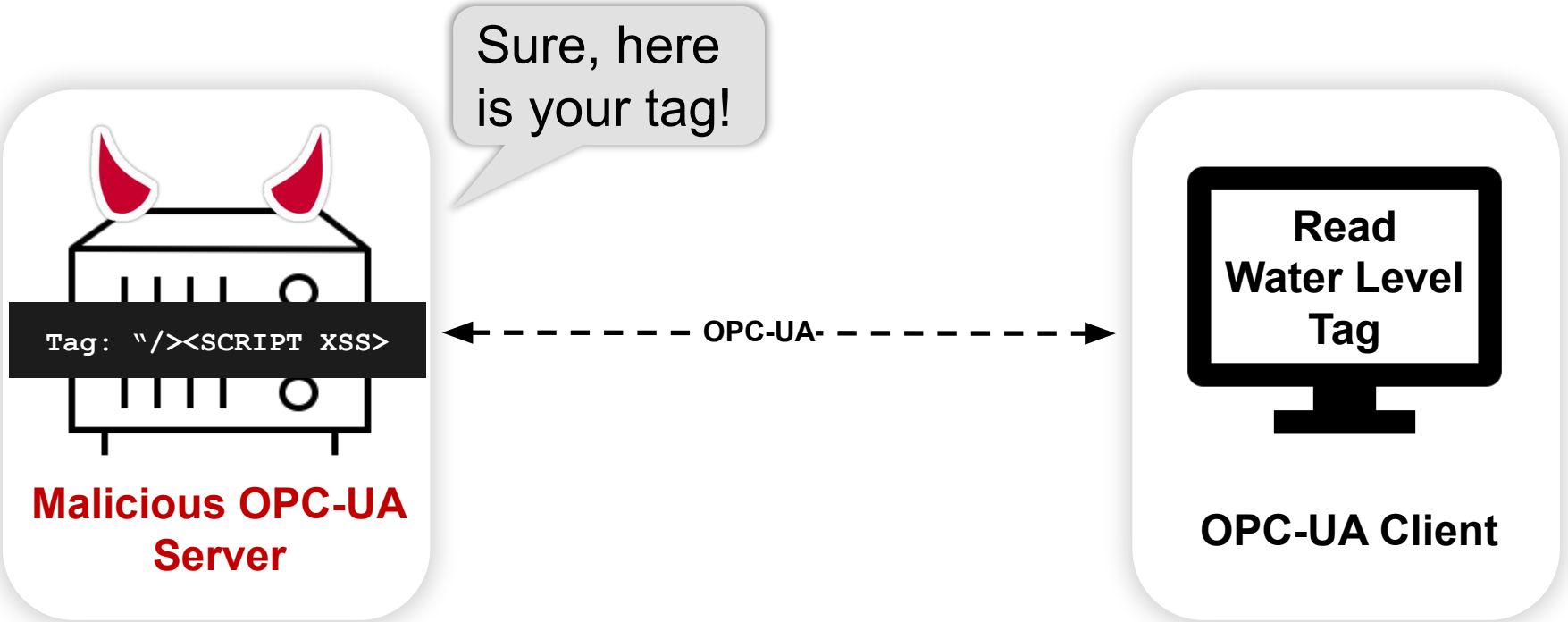
XSS Over OPC-UA



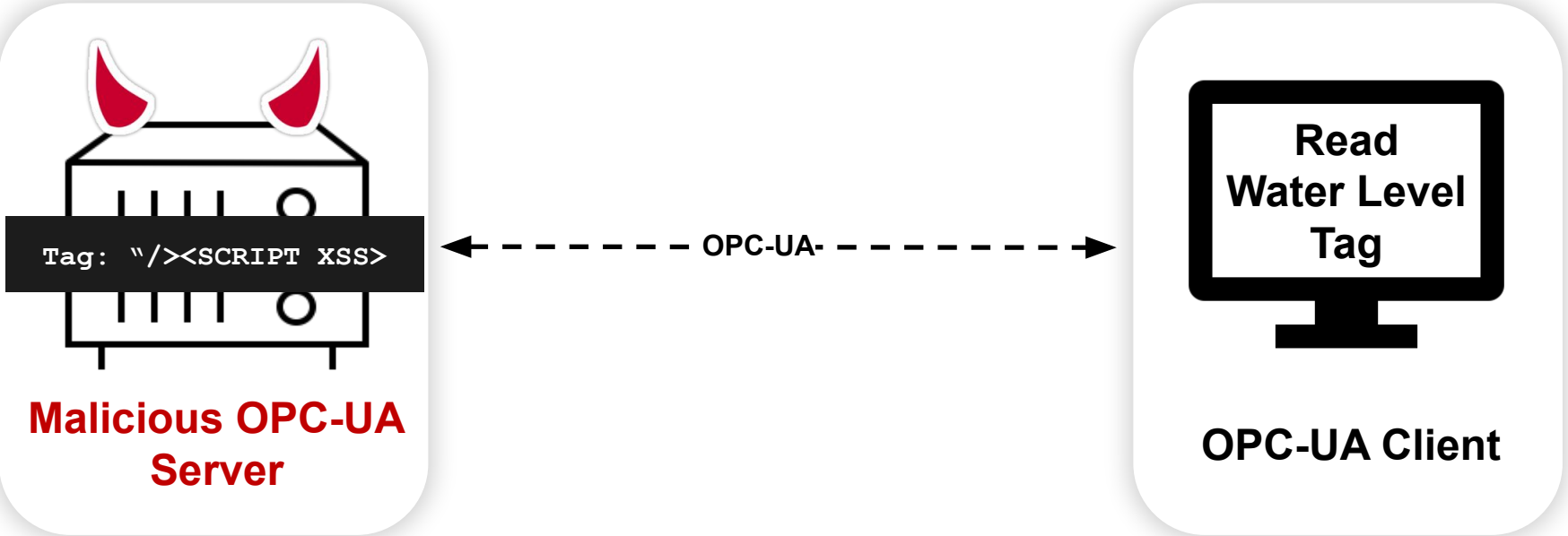
XSS Over OPC-UA



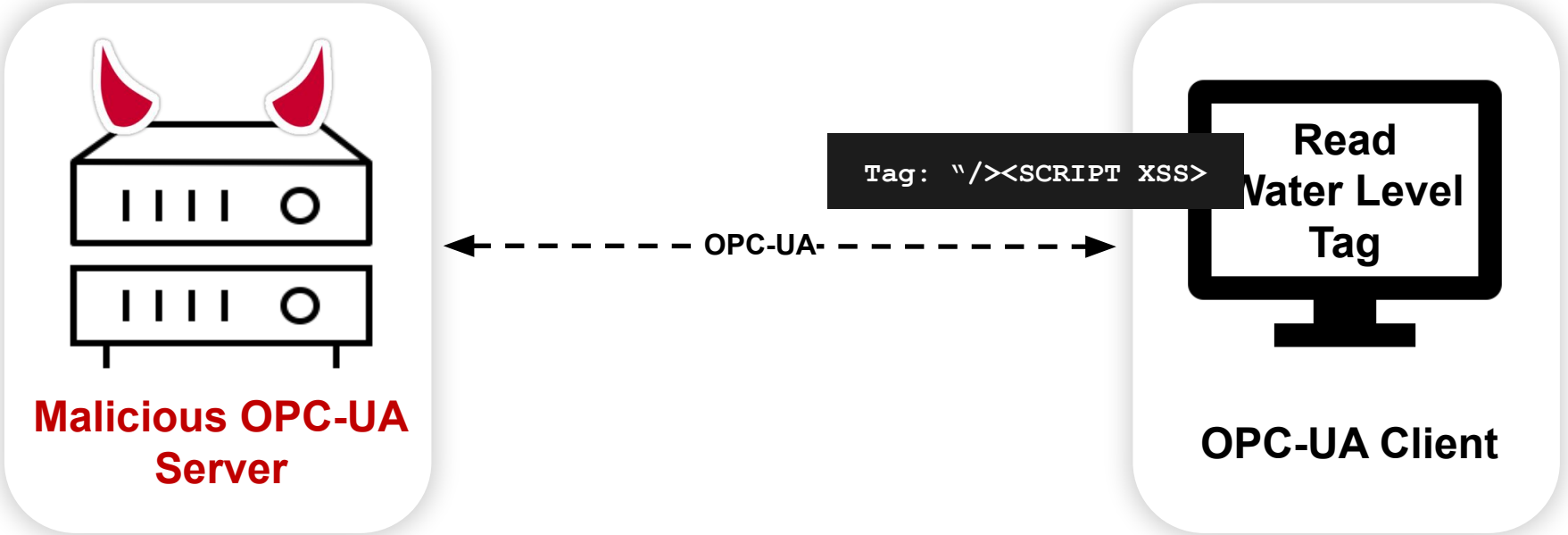
XSS Over OPC-UA



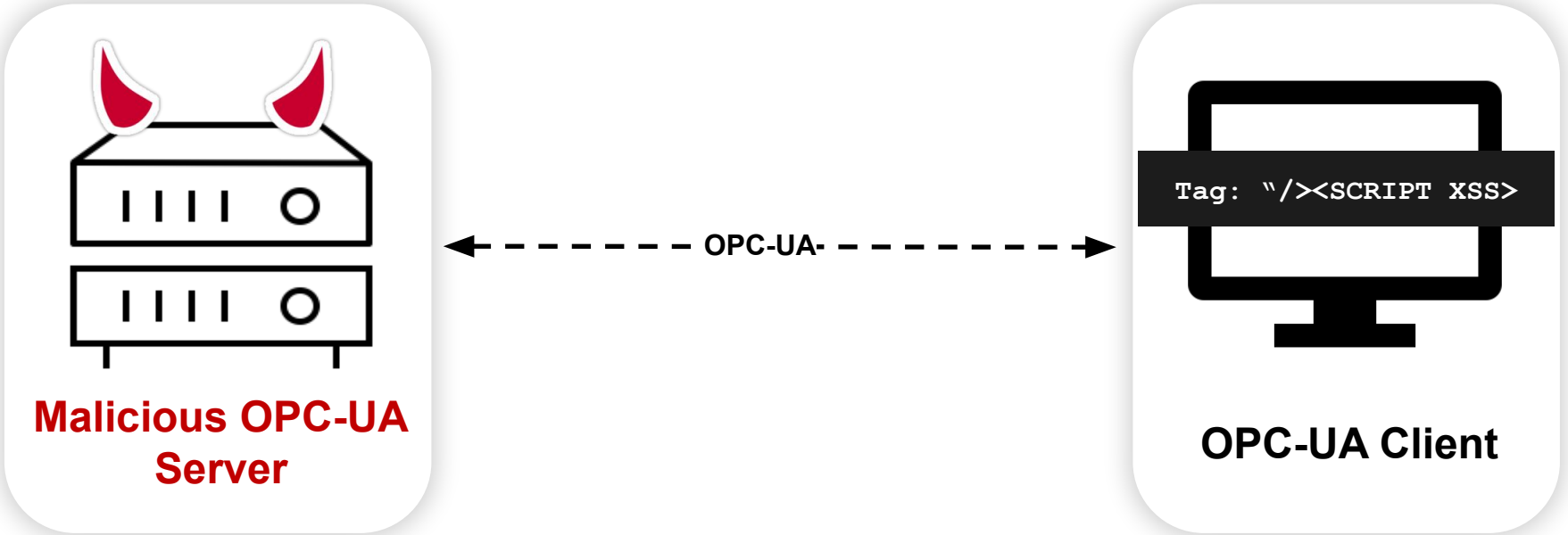
XSS Over OPC-UA



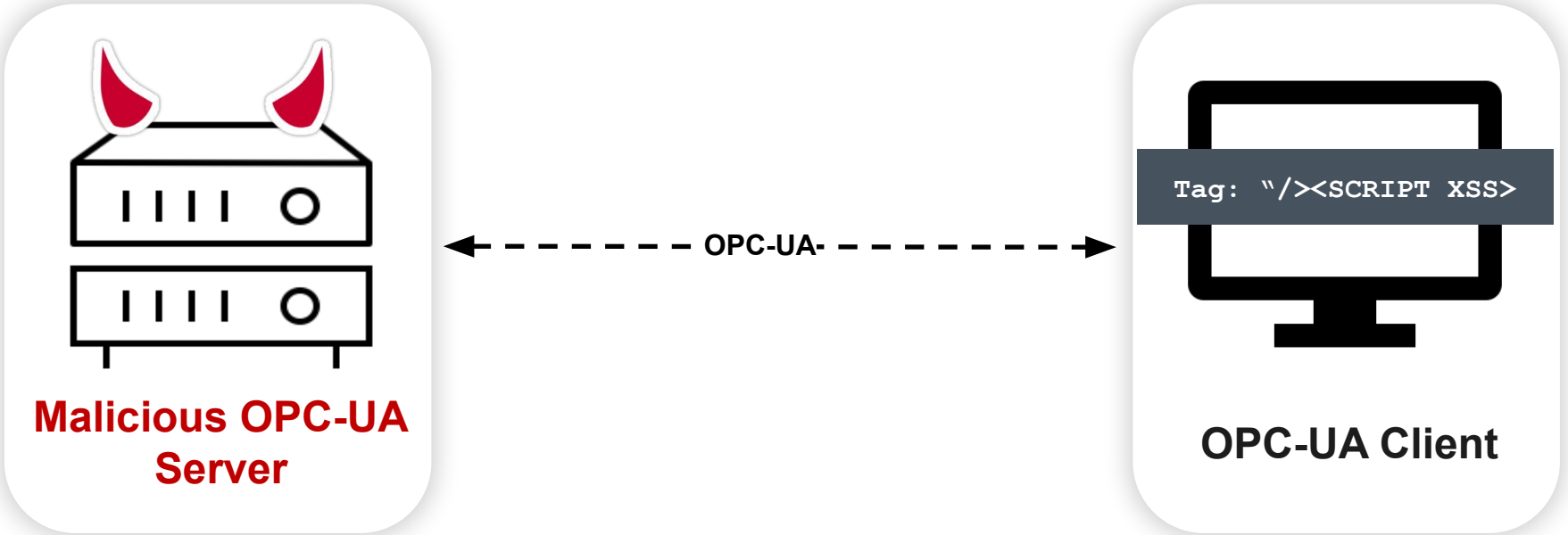
XSS Over OPC-UA



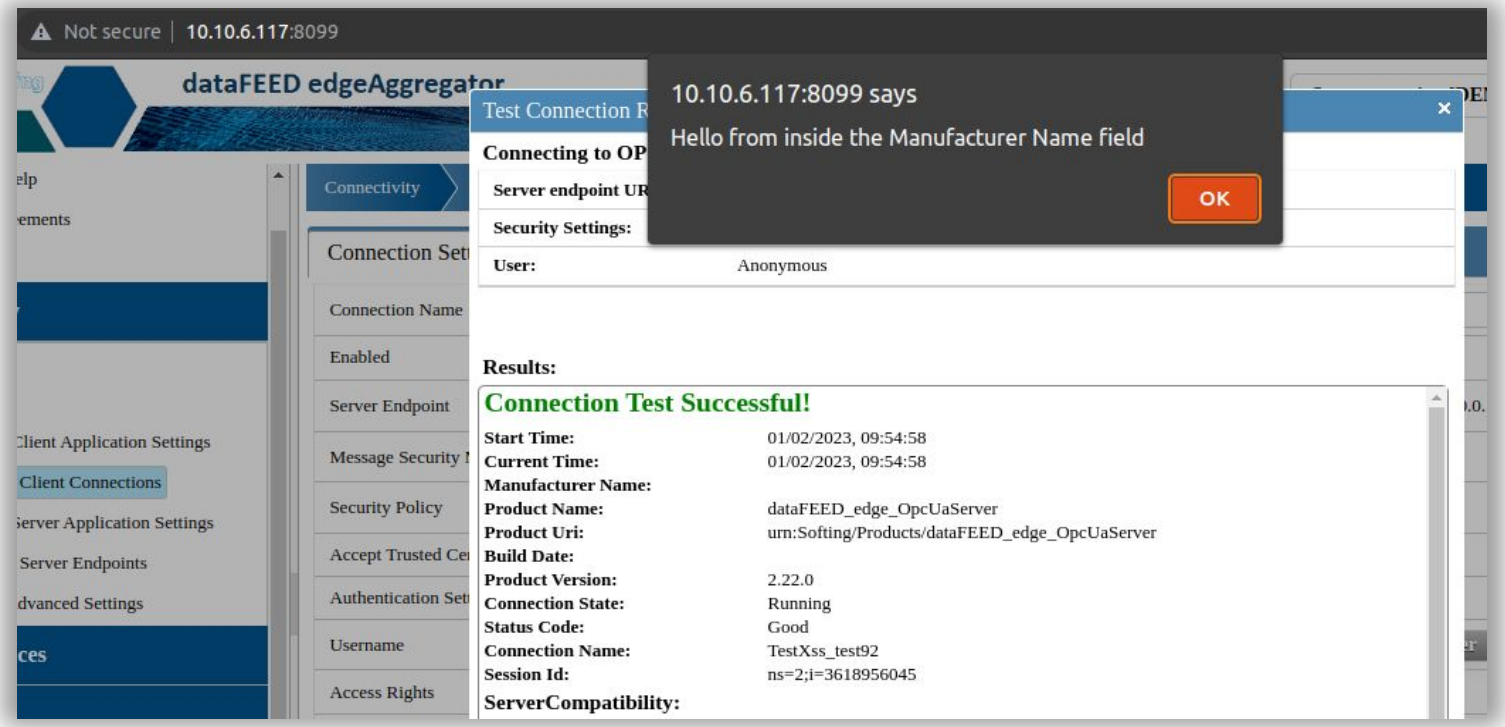
XSS Over OPC-UA



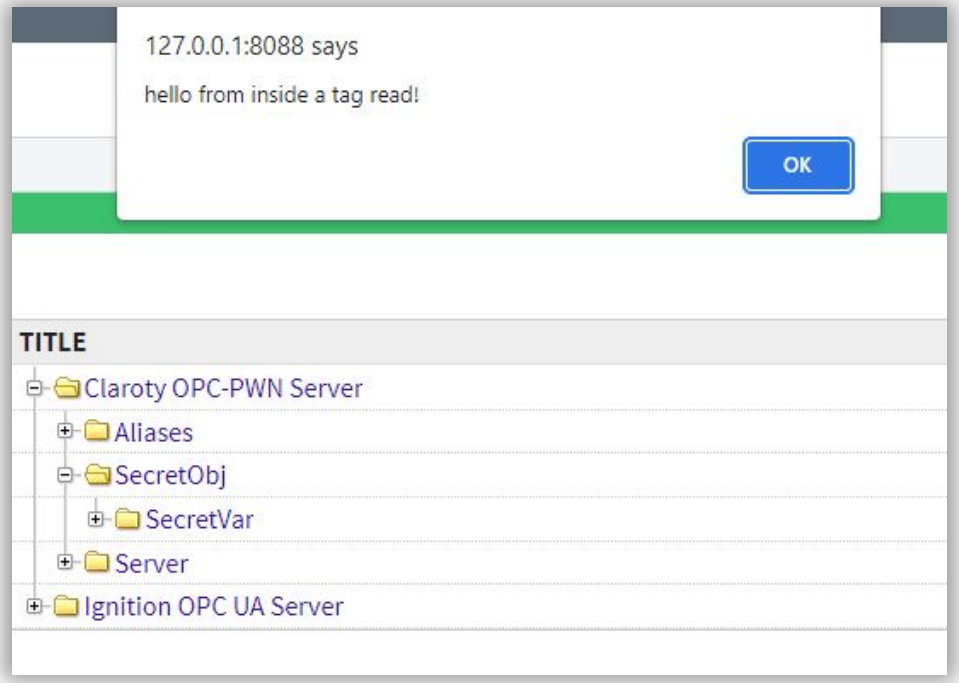
XSS Over OPC-UA



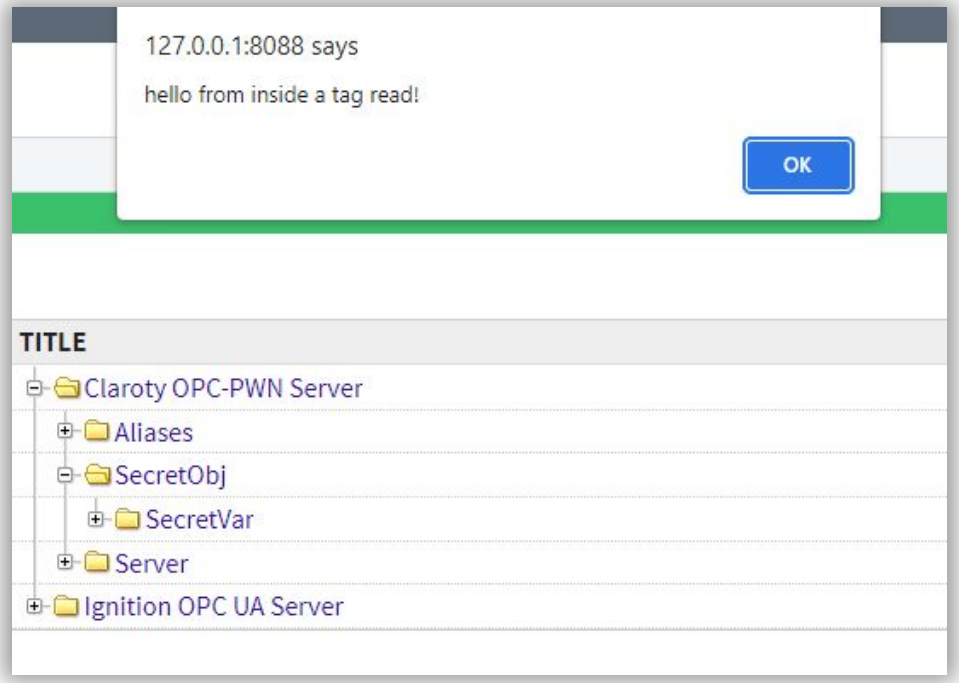
XSS Over OPC-UA

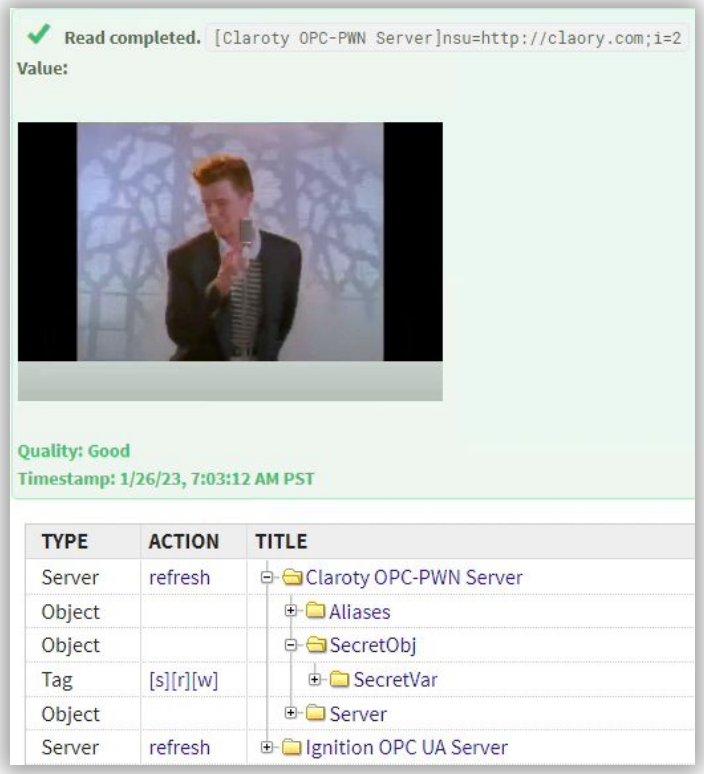


XSS Over OPC-UA



XSS Over OPC-UA



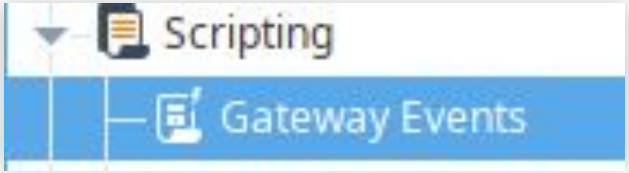


XSS Over OPC-UA to RCE

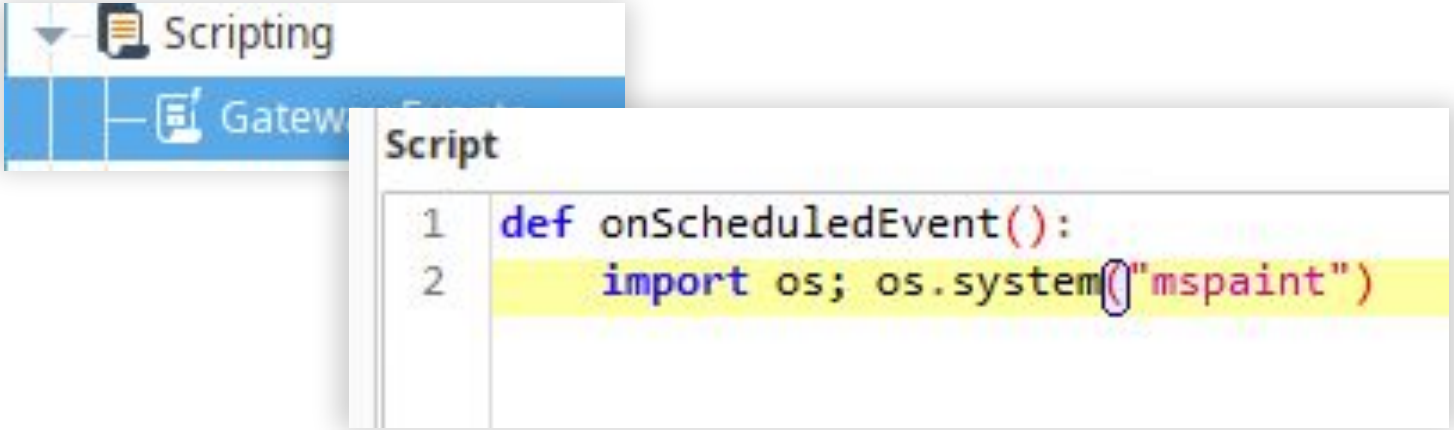
We are in the context of the OPC-UA client, how can we leverage into RCE?

Chain with more vulnerabilities - load a project

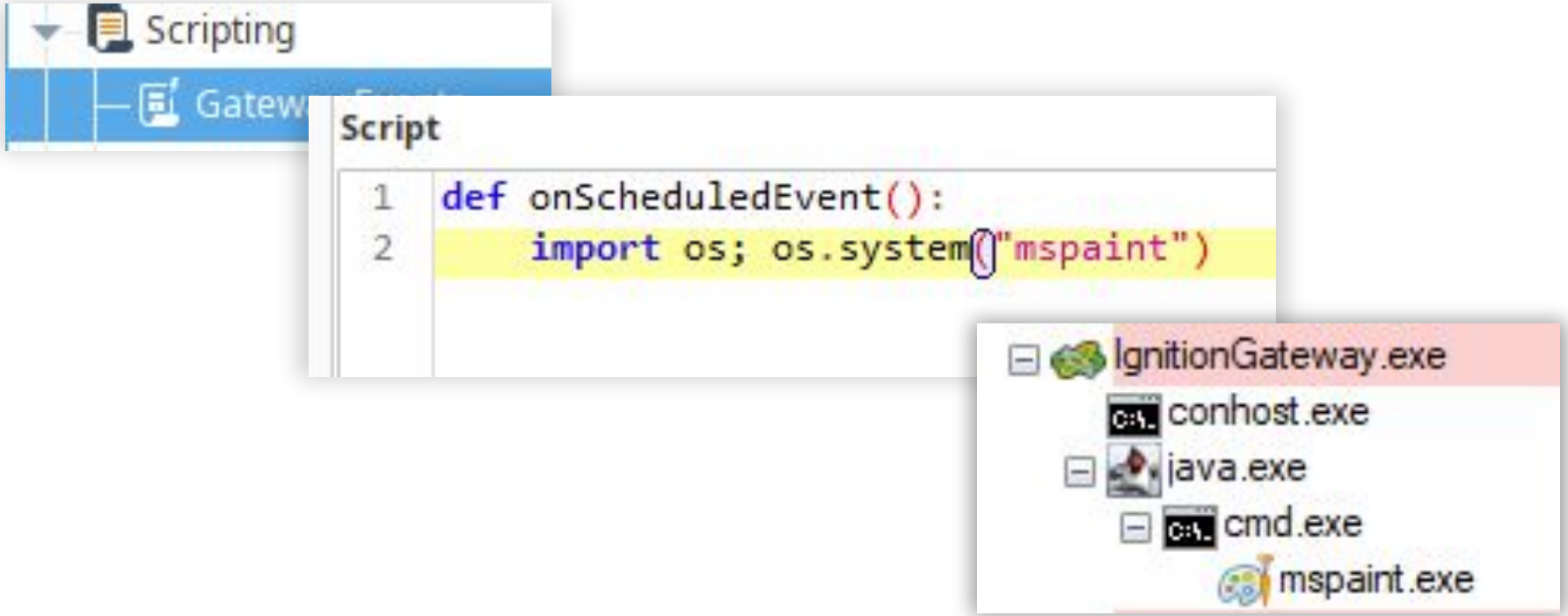
XSS Over OPC-UA to RCE



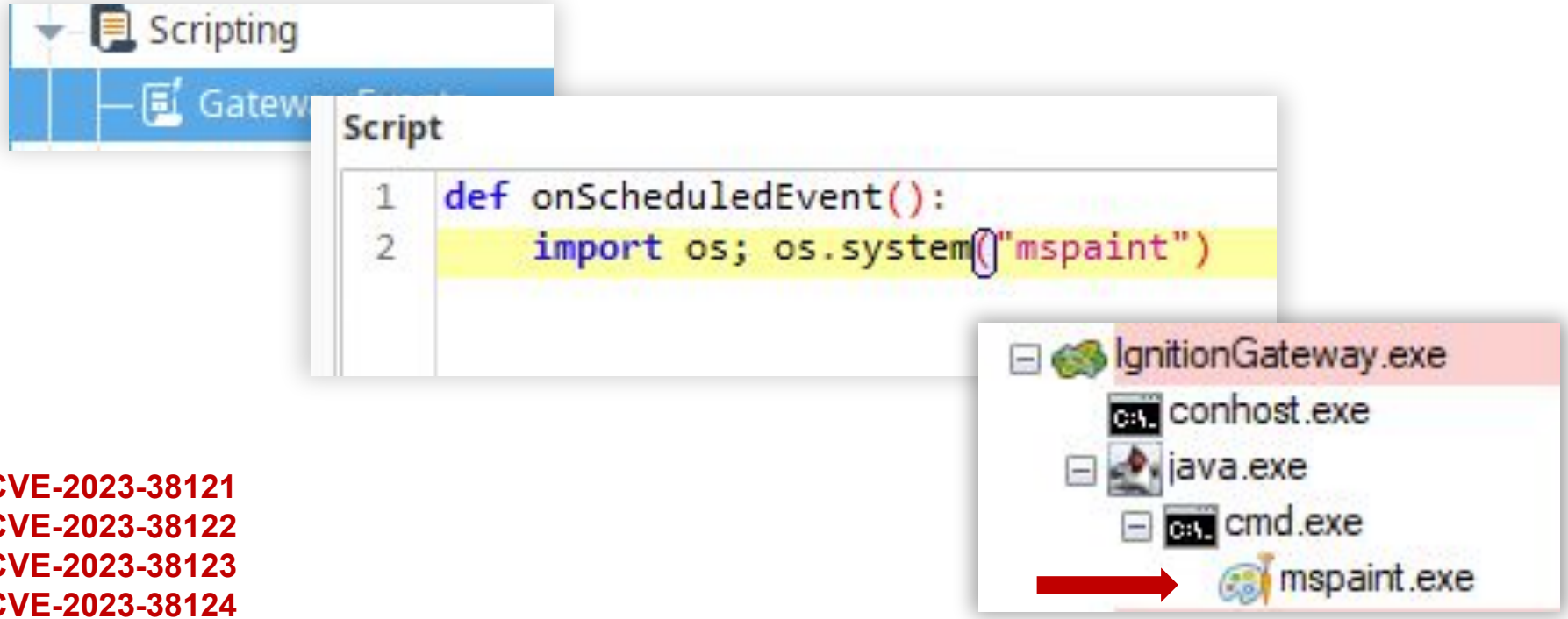
XSS Over OPC-UA to RCE



XSS Over OPC-UA to RCE



XSS Over OPC-UA to RCE



CVE-2023-38121
CVE-2023-38122
CVE-2023-38123
CVE-2023-38124

Pwn2Own Impact on ICS Industry

Takeaways - Pwn2Own is great!

- Uncovers Critical ICS Vulnerabilities
- Drives Vendor Accountability
- Advances Industry Collaboration

Questions?

claroty.com/team82